

第 25 章: Module Odds and Ends (模块的边角细节)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 第五部分(模块与包)的收官之章。前面几章已经把模块的"主菜"上齐了——导入机制、编码基础、包导入——本章则端上一盘"杂烩": 数据隐藏、future、name 双模式技巧、按字符串导入、模块自省、传递式重载(transitive reloads), 外加全套"模块陷阱"(gotchas)。本章内容有些偏高级、可跳过, 但 name 技巧等知识点极其常用, 是进入下一部分"类与面向对象"(Part VI) 前必须浏览的一章。学完本章, 你对 Python 模块体系的理解才算完整。

25.1 开篇引言

英文原文

This chapter concludes this part of the book with an assortment of module-related topics—data hiding, the future module, the name variable, name-string imports, the getattr hook, transitive reloads, and more—along with the usual set of gotchas and exercises rela

ted to what we've covered in this part of the book. Along the way, we'll build some useful tools that combine functions and modules. Like functions, modules are more effective when their interfaces are well-defined, so this chapter also briefly reviews module design concepts. Though some coverage here might qualify as advanced and optional, this is mostly a miscellany of additional module subjects. Because some of the topics discussed here are very widely used—especially the name dual-mode trick—be sure to browse here before moving on to classes in the next part of the book.

中文翻译

本章用一堆与模块相关的主题为本书这一部分收尾——数据隐藏 (data hiding)、future 模块、name 变量、按名字字符串导入 (name-string imports)、getattr 钩子、传递式重载 (transitive reloads) 等等——以及本部分内容常见的一系列陷阱 (gotchas) 和练习。一路走来，我们会构建一些把函数与模块结合起来的实用工具。和函数一样，模块在接口定义良好的时候更高效，所以本章还会简要回顾模块设计概念。尽管本章有些内容可能称得上高级、可选，但大部分只是额外的模块主题杂谈。由于本章讨论的某些主题应用非常广泛——尤其是 name 双模式 (dual-mode) 技巧——在进入下一部分学习类 (class) 之前，务必先浏览本章。

深度理解

·核心概念：本章是"模块三部曲"（22 导入机制、23 编码基础、24 包导入）之后的"补漏章"——把零散的、常用的、奇怪的知识点集中处理，同时用大型案例（reloadall.py）把前面学过的工具（递归、dict、type、name）串起来。

·为什么放在这里：因为下一部分就要讲类（class），而很多模块技巧（getattr、dir）的完整实现要依赖类的知识。作者特意提醒：name 双模式技巧太常用了，不看会亏。

·生活例子：前面几章像是学"怎么把房间（模块）盖起来、怎么装门牌（导入路径）"，这一章则是学"房间里的暗格（数据隐藏）、多功能家具（双模式代码）、监控摄像头（内省工具）"——东西琐碎，但都是生活必需品。

·初学者误区：误以为模块知识到此为止、"导入就是 import 那么简单"。实际上 from 的污染问题、reload 与 from 的兼容性问题，都是实战中会真实踩到的坑。

25.2 模块设计概念 (Module Design Concepts)

英文原文

First up, some perspective. Like functions, modules present design trade-offs: you have to think about which functions go in which modules, module communica

tion mechanisms, and so on. All of this will become clearer when you start writing bigger Python systems, but here are a few general ideas to keep in mind: You're always in a module in Python. There's no way to write code that doesn't live in some module. As mentioned briefly in Chapters 17 and 21, even code typed at the interactive prompt (a.k.a. REPL) really goes in a built-in module called `main`; the only unique things about the interactive prompt are that code runs and is discarded immediately, and expression results are printed automatically.

Minimize module coupling: global variables. Like functions, modules work best if they're written to be mostly closed boxes. As a rule of thumb, they should be as independent of global variables used within other modules as possible, except for functions and classes imported from them. The only things a module should share with the outside world are the tools it uses, and the tools it defines.

Maximize module cohesion: unified purpose. Also like functions, you can minimize a module's couplings by maximizing its cohesion. If all the components of a module share a general purpose, they're less likely to depend on external names. Modules should rarely change other modules' variables. We illustrated this with code in Chapter 17, but it's worth repeating here: it's perfectly OK to use globals defined in another module (that's how clients import services, after all), but changing globals in another module is often a symptom of

a design problem. There are exceptions, of course, but you should try to communicate results through devices such as function arguments and return values, not cross-module changes. Otherwise, your globals' values become dependent on the order of arbitrarily remote assignments in other files, and your modules become harder to understand and reuse. As a summary, Figure 25-1 sketches the environment in which modules operate. Modules contain variables, functions, and classes, and import other modules for the tools they define. Functions have local variables of their own, as do classes—objects that live within modules and which we'll begin studying in the next chapter. As we saw in Part IV, functions can nest, too, but all are ultimately contained by modules at the top.

中文翻译

首先，一些背景视角。和函数一样，模块也面临设计上的权衡取舍：你必须考虑哪些函数放进哪个模块、模块之间如何通信等等。当你开始编写更大的 Python 系统时，这一切会变得更加清晰，但这里先给出几条要记住的通用原则：在 Python 中，你永远处于某个模块之中。没有任何办法写出不属于某个模块的代码。正如第 17 章和第 21 章简要提到的，即使在交互式提示符（也就是 REPL）下输入的代码，实际上也存放在一个名为 `main` 的内置模块里；交互式提示符唯一的特别之处在于：代码立即运行并被丢弃，表达式结果会自动打印。最小化模块耦合：全局变量。和函数一样，模块在尽量写成“封闭盒子”时工作得最好。经验法则：模块应

尽可能不依赖其他模块中使用的全局变量，唯一的例外是从它们那里导入的函数和类。一个模块应该与外部世界共享的，只有它使用的工具，以及它定义的工具。最大化模块内聚：统一的目的。同样和函数一样，可以通过最大化内聚性来最小化模块的耦合。如果模块的所有组成部分都服务于一个总体目的，它们就不太可能依赖外部名字。模块应该很少修改其他模块的变量。我们在第 17 章用代码演示过这一点，但值得在此重复：使用另一个模块中定义的全局变量完全没问题（毕竟客户端就是这样导入服务的），但修改另一个模块中的全局变量通常是设计问题的征兆。当然也有例外，但你应该尽量通过函数参数、返回值这类机制来传递结果，而不是跨模块修改。否则，你的全局变量值将取决于其他文件中任意遥远赋值语句的执行顺序，你的模块会变得难以理解和复用。作为总结，图 25-1 勾勒了模块运行的环境。模块包含变量、函数和类，并导入其他模块以获得它们定义的工具。函数有自己的局部变量，类也一样——类是生活在模块内部的对象，我们将在下一章开始学习。如我们在第四部分所见，函数还可以嵌套，但归根结底，所有一切都由顶层的模块所容纳。

深度理解

·核心概念：模块设计的两大支柱——内聚（cohesion）与耦合（coupling）。内聚指模块内部各部分是否“劲儿往一处使”；耦合指模块之间互相依赖的程度。好设计 = 高内聚 + 低耦合。

·“你永远处于模块之中”：这是 Python 与其他语言的根本差异之一。REPL 输入 `x = 1`，其实是在 `main` 模块的命名空间里赋值；所有顶层代码都栖身于某个模块。所以 `nam`

e 这个变量才无处不在。

·设计原因：Python 是动态语言，模块之间通过名字（namespace）通信。如果 A 模块随手改 B 模块的全局变量，程序的执行结果就取决于“谁先运行”——这种隐式时序依赖是 bug 的温床。把通信收敛到函数参数和返回值，程序的行为才是确定的。

·生活例子：耦合像合租——室友（模块）之间如果随意翻动对方的私人物品（改对方全局变量），生活秩序就乱了；约定“各管各的房间、公共区域（函数接口）协商使用”，才相安无事。内聚像抽屉分类——一个抽屉只放一类东西（一个模块只干一类事），找起来才快。

·初学者误区：以为“import 别人的模块就可以随便改里面的变量”。可以改，但“能做”不等于“该做”——这会制造难以调试的隐式依赖。

25.3 模块中的数据隐藏 (Data Hiding in Modules)

英文原文

Next, we turn to private matters. As we've seen, a Python module exports all the names assigned at the top level of its file. There is no syntax for declaring which names should and shouldn't be visible outside the module. In fact, there's no way to prevent a client from changing names inside a module if it wants to. In Python, data hiding in modules is a convention, not a

syntactical constraint. If you want to break a module by trashing its names, you can, but most programmers don't count this as a life goal. Some purists object to this liberal attitude toward data hiding, claiming that it means Python can't implement encapsulation. However, encapsulation in Python is more about packaging than about restricting. We'll expand on this idea in the next part in relation to classes, which also have no privacy syntax but can often emulate its effect in code.

中文翻译

接下来，我们转向“隐私”问题。如我们所见，Python 模块会导出文件顶层赋值的所有名字。没有任何语法可以声明哪些名字应该在模块外部可见、哪些不可见。事实上，如果客户端执意要修改模块内部的名字，没有任何办法能阻止。在 Python 中，模块的数据隐藏是一种约定（convention），而不是语法约束。如果你想通过篡改名字来搞坏一个模块，你完全可以做到，但大多数程序员不会把这当作人生目标。一些纯粹主义者反对这种对待数据隐藏的宽松态度，声称这意味着 Python 无法实现封装（encapsulation）。然而，Python 中的封装更多关乎“打包”（packaging），而非“限制”（restricting）。我们将在下一部分结合类来展开这个观点——类同样没有隐私语法，但通常可以在代码中模拟出隐私的效果。

深度理解

·核心概念：模块数据隐藏 = 约定而非语法。X 下划线只

是"礼貌性提示", Python 解释器不会强制执行任何访问限制。

·底层实现: 模块的所有顶层名字都存储在模块对象的 dict 字典里, 任何 import 进来的代码都能通过 module.dict 或属性访问直接读写。解释器层面根本没有"私有"标记。

·设计原因: Python 哲学是"我们都是一群成年人" (We are all consenting adults) ——信任程序员不会乱来, 同时把权力留给需要的人 (比如调试时直接改模块内部状态)。封装的重点是"把相关的名字打包在一起", 而不是"挡住外人"。

·生活例子: 约定式隐私像"办公室文化"——同事桌上的文件夹 (X) 标着"内部资料", 大家自觉不动; 但办公室没有上锁的门, 真要看也没有物理障碍。信任 + 自觉, 而不是铁锁。

·初学者误区: 从 Java/C++ 过来的学习者常问"Python 的 private 在哪?"——没有。X 不是 private, X (双下划线, 下一部分讲类时会遇到) 也只是名字改写 (name mangling), 依然能访问。真正的防护在人的约定里, 不在语法里。

25.4 最小化 from 的破坏: X 与 all (Minimizing from Damage: X and all)

英文原文

That being said, as a limited special case, you can prefix names with a single underscore (e.g., `_X`) to prevent them from being copied out when a client imports a module's names with a `from` statement. This really is intended only to minimize namespace pollution; because `from` copies out all names, the importer may get more than it's bargained for (including names that overwrite names in the importer).

But underscores aren't "private" declarations: you can still see and change such names with other import forms. Example 25-1 demos the idea.

Example 25-1. `unders.py`

```
python a, b, c, d = 1, 2, 3, 4 # Control from exports, take 1
```

When names both with and without underscores are assigned this way, `from` can't see the former, but `import` and normal `from` can:

```
python $ python3 >> from unders import # Load non-X names only
>> from unders import a, b (1, 2) >> c
NameError: name 'c' is not defined. Did you mean:''?
```

```
>> from unders import c # But other importers get every name
>> c 3 >> import unders >> unders.d 4
```

Alternatively, you can achieve a hiding effect similar to the `X` naming convention by assigning a list of vari

able name strings to the variable all at the top level of the module. When this feature is used, the from statement will copy out only those names listed in the all list, though other imports work as before.

In effect, this is the converse of the X convention: all identifies names to be copied, while X identifies names not to be copied. Python looks for an all list in the module first, and copies its names irrespective of any underscores; if all is not found, from copies all names without a single leading underscore. To demo, Example 25-2 uses both name-hiding tools.

Example 25-2. alls.py

```
python all = ['a','c'] # Control from exports, take 2 a,
b, c, d = 1, 2, 3, 4 # all has precedence over X
```

On imports, from gets everything in all, but no others; other importers again get everything:

```
python $ python3 >> from alls import # Load all names - only
>> a, c # Even if they have underscores (1, 3)
>> b NameError: name'b' is not defined >> from alls import a, b, c, d # But other importers get every name
>> a, b, c, d (1, 2, 3, 4) >> import alls >> alls.a, alls.b, alls.c, alls.d (1, 2, 3, 4)
```

Like the X convention, the all list has meaning only to the from statement form and does not amount to a privacy declaration: other import statements can still access all names, as the last two tests show. Still, mo

module writers can use either technique to implement modules that are well-behaved when used with from .

See also the discussion of all lists in package init.py files in Chapter 24. In this context, these lists declare nested submodules to be automatically loaded for a from run on their container. The effect is similar to name hiding in module files, though packages extend it to apply to the content of a package folder in the file system.

中文翻译

话虽如此，作为有限的特殊情况，你可以给名字加上单个下划线前缀（例如 X），这样当客户端用 from 语句导入模块的名字时，这些名字就不会被复制出去。这真的只是用来最小化命名空间污染（namespace pollution）：因为 from 会复制出所有名字，导入方可能得到超出预期的东西（包括覆盖导入方已有名字的名字）。但下划线并不是“私有”声明：用其他导入形式，你依然能查看和修改这些名字。示例 25-1 演示了这一思想。示例 25-1. unders.py

```
python a, b, c, d = 1, 2, 3, 4 # 控制 from 的导出，第一招当带下划线和不带下划线的名字以这种方式赋值后，from 看不到前者，但 import 和普通 from 都能看到：python $ python3
>>> from unders import # from 只加载非 X 名字>>>
a, b (1, 2) >>> c NameError: name 'c' is not defined. Did you mean:''? >>> from unders import c # 但其他导入方式能拿到所有名字>>> c 3 >>> import unders >>
> unders.d 4 或者，你也可以在模块顶层给变量 all 赋一个名字字符串列表，从而获得与 X 命名约定类似的隐藏效
```

果。使用该特性后，from 语句只会复制出 all 列表中的那些名字，其他导入方式照旧工作。实际上，这与 X 约定正好相反：all 指明要被复制的名字，而 X 指明不要被复制的名字。Python 会先在模块中查找 all 列表，若找到则无论名字有没有下划线都照单复制；若找不到 all，from 就复制所有不带单个前导下划线的名字。为了演示，示例 25-2 同时用到了这两种名字隐藏工具。示例 25-2. alls.py

```
python
all = ['a','c'] # 控制 from 的导出，第二招
a, b, c, d = 1, 2, 3, 4 # all 优先于 X 约定在导入时，from 只拿到 all 里列出的名字，其他的一概不取；其他导入方式依然能拿到全部
: python $ python3 >>> from alls import # 只加载 all 里的名字
>>> a, c # 即使它们带下划线(1, 3)
>>> b
NameError: name 'b' is not defined
>>> from alls import a, b, c, d # 但其他导入方式能拿到所有名字
>>> a, b, c, d
(1, 2, 3, 4)
>>> import alls
>>> alls.a, alls.b, alls.c, alls.d
(1, 2, 3, 4)
```

和 X 约定一样，all 列表只对 from 语句形式有意义，并不构成隐私声明：其他导入语句依然可以访问所有名字，正如最后两个测试所示。不过，模块作者可以用这两种技巧中的任意一种，来实现对 from 友好的、行为良好的模块。另请参见第 24 章关于包 init.py 文件中 all 列表的讨论。在那个场景下，这些列表声明了嵌套子模块，使得对容器执行 from 时自动加载它们。效果与模块文件中的名字隐藏类似，只是包把它扩展到了文件系统里整个包文件夹的内容。

深度理解

·核心概念：控制 from 导出的两条机制：X 下划线前缀（黑名单思维：藏住这些）与 all 列表（白名单思维：只给

这些)。两者冲突时，all 优先。

- 底层实现：from 由 import 系统实现：解释器检查模块是否定义了 all 属性（在模块对象的 dict 中查找），有则按列表复制；没有则遍历 dict 过滤掉单个下划线开头的名字。注意 import 与普通 from 完全不参与这个过滤逻辑。

- 设计原因：from 的语义是"把命名空间合并进来"，最容易造成名字冲突（你模块里的 print、len 若被覆盖就出大事了）。给模块作者一个"收敛出口"的开关，是 Python 在自由与秩序之间找的平衡点。

- 生活例子：X 像"员工手册里写着带下划线的文件不外传"的办公室惯例；all 像公司前台摆放的"访客宣传册"——访客（from）只能拿到前台摆出来的那几本，但员工（import）进资料室想拿什么拿什么。

- 初学者误区：把 X 和 all 当成"私有声明"，以为外部真的访问不到。它们只影响 from 这一个语句；用 import 照样拿得到。真正的私有控制要靠下一部分学的类 + 属性钩子。

- 现代实践：在库的 init.py 中写 all 是标准做法——它不仅控制 from，还被 IDE、代码检查工具和 help() 用来判断"公开 API 是什么"。

代码分析 (Example 25-1 与 25-2)

```
a, b, _c, _d = 1, 2, 3, 4      # from *
```

- 解释器如何处理：这条语句执行时，Python 把 a、b、c、d 四个名字绑定到对应整数对象，存入模块 dict。当外

部执行 `from unders import` 时，解释器遍历该字典，只复制不带单个前导下划线的名字——`a`、`b` 被复制，`c`、`d` 被跳过。

·设计思想：`X` 是“隐式约定”，`all` 是“显式清单”——Python 之禅说“显式优于隐式”，`all` 正是更显式的进化方向。

25.5 管理属性访问：getattr 与 dir (Managing Attribute Access: getattr and dir)

英文原文

On the subject of data hiding in modules, Python 3.7 added support for special functions at a module's top level that can be used to manage access to a module's attributes. If defined, a module's `getattr` function is automatically run when a module attribute is not found, and its `dir` overrides the normal attribute-list fetch run for the `dir` built-in. These can be used to implement both basic access constraints and arbitrarily dynamic interfaces.

These functions also shadow same-named tools in classes and are meant in part to obviate a long-standing and obscure trick that reset a module's object in the `sys.modules` table to an instance of a class with the same methods. This, of course, means that these functions may make more sense after we study classes in Part VI, but the artificial module in Example 25-3

demos the basics.

Example 25-3. gamod.py

```
python var = 2 # Real attribute returned directly def
getattr(name): # Undefined attr fetches routed here
print(f'(virtual {name})', end='') match name: case'tes
t':
```

```
return name var case'hack' |'code':
```

```
return name.upper() case :
```

```
raise AttributeError(f'{name} is undefined') def dir(): r
eturn ['var','test','hack','code'] When imported, fetches
of real attributes defined in the module work normally (subject to the X and all of the prior section for from ), but missing names are routed to getattr, which can manage the request. It may also use a raise statement to flag an invalid request with an exception—a topic we'll study in full later in the book because it's also dependent on classes today:
```

```
python >> import gamod >> gamod.var # Real: geta
ttr not called 2 >> gamod.test # Virtual: computed w
hen fetched (virtual test)'testtest'>> gamod.hack (virt
ual hack)'HACK'>> gamod.nonesuch AttributeError: n
onesuch is undefined >> dir(gamod) ['code','hack','te
st','var']
```

The from statement invokes getattr too, though from requires names to be listed on all (you can largely ignore the spurious path fetch here, though a getattr

must accommodate it):

```
python >> from gamod import code (virtual path) (virtual code) >> code'CODE'>> from gamod import (virtual path) (virtual all) >> var 2 >> hack
NameError: name'hack' is not defined
```

Importantly, `getattr` is not run for global-scope look up within the module itself, so in-file undefined names remain undefined. It's really just for attribute fetches from other modules and does not catch assignments anywhere. For example, the first line of the following added at the bottom of Example 25-3 would fail, and the second line run in the REPL would make a new attribute in the module which bypasses `getattr` the reafter:

```
python print(test) # File: does NOT call getattr (raises NameError)
gamod.hack = 'real' # REPL: does NOT call getattr (makes attribute)
Although this all works as advertised, it is a tool-builder's hook, and you'll have to unearth legitimate use cases. It may be useful in narrow roles, but it also conflates modules with classes and discounts the fact that module learners do not already understand these functions' origins in classes.
```

This is a regrettably common theme in Python: additions often come with forward dependencies that seem to expect users to already know Python in order to use Python. Python is not just for Python experts, but that's a message baked into many a mod.

The good news here may be that a later proposal to add classes' `setattr` for module-attribute assignment was rejected by Python's steering committee—though only after allowing `getattr` and `dir` to sneak in. As usual, you should weigh the convolutions of this extension against its real-world applications.

中文翻译

说到模块中的数据隐藏，Python 3.7 增加了对模块顶层特殊函数的支持，可以用来管理对模块属性的访问。如果定义了，当模块属性找不到时，模块的 `getattr` 函数会被自动调用；而 `dir` 会覆盖 `dir` 内置函数运行时正常的属性列表获取。这两个函数既可以实现基本的访问约束，也可以实现任意动态的接口。这两个函数也会遮蔽（shadow）类中同名工具，部分目的是淘汰一个历史悠久又晦涩的技巧——该技巧把 `sys.modules` 表中模块的对象替换成带有这些方法的类的实例。当然，这意味着这两个函数在我们学过第六部分的类之后才更有意义，不过示例 25-3 里的人造模块演示了基础用法。示例 25-3. `gamod.py`

```
python var = 2 # 真实属性，直接返回
def getattr(name): # 未定义的属性获取被路由到这里
    print(f'(virtual {name})', end='')
    match name:
        case 'test': return name
        case 'hack' | 'code': return name.upper()
        case : raise AttributeError(f'{name} is undefined')
    def dir(): return ['var', 'test', 'hack', 'code']
```

导入后，访问模块中已定义的真实属性工作正常（对 `from` 而言受上一节 `X` 和 `all` 约束），但缺失的名字会被路由到 `getattr`，由它来管理请求。它还可以用 `raise` 语句通过异常（exception）标记非法请求——这个主题本书后面会完整

讲授，因为它今天也依赖于类的知识：python >>> import gamod >>> gamod.var # 真实属性：不调用 getattr 2 >>> gamod.test # 虚拟属性：获取时才计算(virtual test)'testtest'>>> gamod.hack (virtual hack)'HACK'>>> gamod.nonesuch AttributeError: nonesuch is undefined >>> dir(gamod) ['code','hack','test','var'] from 语句同样会调用 getattr，不过 from 要求名字列在 all 中（这里那一次多余的 path 获取可以基本忽略，但 getattr 必须兼容它）：python >>> from gamod import code (virtual path) (virtual code) >>> code'CODE'>>> from gamod import (virtual path) (virtual all) >>> var 2 >>> hack NameError: name'hack' is not defined 重要的是，getattr 不会在模块内部的全局作用域查找中运行，因此文件内的未定义名字仍然是未定义的。它真的只服务于其他模块发起的属性获取，而且在任何地方都不拦截赋值。例如，把下面第一行加到示例 25-3 的底部会失败，而在 REPL 中运行第二行则会在模块里新建一个属性，从此绕过 getattr：python print(test) # 文件内：不会调用 getattr（抛 NameError）gamod.hack = 'real' # REPL 中：不会调用 getattr（直接建属性）虽然这一切都如描述般工作，但它是一个“造工具者”的钩子，你得自己去发掘合理的用例。它在窄小的场景下也许有用，但它把模块与类混为一谈（conflates），并且忽略了这样一个事实：模块的学习者并不理解这些函数在类中的出身。这是 Python 中一个令人遗憾的常见主题：新特性往往带着前向依赖，仿佛期待用户为了用 Python 而必须先懂 Python。Python 不只是为 Python 专家准备的，但这句话被烙进了许多 mod（模块）之中。这里的好消息或许是：后来一个“给模块属性赋值增加类

的 `setattr` 的提案被 Python 的指导委员会 (steering committee) 否决了——尽管是在允许 `getattr` 和 `dir` 溜进来之后。和往常一样，你应该权衡这个扩展的绕弯程度与其真实世界应用的价值。

深度理解

- 核心概念：模块级的 `getattr` / `dir` 钩子——属性找不到时“外包”给一个函数处理，`dir()` 的结果也可以自定义。这是 Python 3.7 的新特性，本质上是把类的魔法方法能力“部分移植”到模块上。

- 底层实现：CPython 的属性访问链：`module.attr` 先查模块对象的 `dict`；查不到时，若模块定义过 `getattr` 则调用它，函数返回什么就得到什么；若它抛出 `AttributeError`，访问就报 `AttributeError`。`dir` 则完全接管 `dir(module)` 的输出。注意：全局作用域内的名字查找（函数体内、模块顶层）走的是另一条路——编译期的名字解析 + 运行时 `LOADGLOBAL` 指令，根本不经 `getattr`。

- 设计原因：历史上有人用“把 `sys.modules['mod']` 替换成类实例”的黑魔法来实现模块虚拟属性。官方觉得这太丑，于是把 `getattr` 和 `dir` 直接搬到了模块顶层。作者吐槽：这又是个“先懂类才能用模块”的前向依赖——新手拿它没什么用。

- 实际问题：典型场景——为大型配置模块实现“虚拟配置项”（访问任意键动态生成）、为懒加载设计占位模块（如数据科学里的延迟导入）、让 `dir()` 只展示公开 API。但绝大多数业务代码用不到。

·初学者误区：以为 `getattr` 能拦截"模块内部的全局变量查找"或"对模块属性的赋值"——都不能。它只管外部对不存在属性的读取。误把 `getattr` 与 `setattr`（类专属）混淆。

·生活例子：`getattr` 像大酒店的"前台总机"——你打内线找不存在的分机（属性），总机会告诉你"这个号码是虚拟的，转接给你对应部门"（动态计算）；但酒店内部员工直接用内网短号找人，根本不经总机（模块内部查找不经过它）。

代码分析 (Example 25-3)

```
var = 2                                #
def __getattr__(name):                 #
    print(f'(virtual {name})', end=' ')
    match name:
        case 'test':
            return name * var          # 'test'
        case 'hack' | 'code':
            return name.upper()        # 'hack' 'code'
        case _:
            raise AttributeError(f'{name} is undefined') #
```

·解释器如何处理：`import gamod` 时模块顶层依次执行：
： `var = 2` 把名字 `var` 存入模块字典；`def getattr` 和 `def dir` 创建函数对象并把名字绑定进字典。之后外部访问 `gamod.test` 时，解释器先在 `gamod.dict` 里找 `test`——找不到，于是调用模块的 `getattr('test')`：先打印 `(virtual test)`，然后 `match name` 匹配到 `case 'test'`，返回 `'test' 2` 即 `'testtest'`。访问 `gamod.nonesuch` 时，`case` 分支抛出 `AttributeError`——这正是 Python 在属性查找失败时

该有的表现。而 `from gamod import code` 内部会先做 `path` 探测（导入系统要确认它是否是个包），所以输出里先出现 (virtual path)。

·设计思想：动态属性 = "接口与数据解耦"。函数式地把"有什么属性"从静态声明变成运行时计算。但要克制：滥用它会让 IDE、静态检查、`help()` 全部失灵，代码不可读。

25.6 启用语言变更：future (Enabling Language Changes: future)

英文原文

Speaking of changes, Python mods that may break existing code are often introduced gradually. This is not always as "gradual" as it might be, and version 3.0 was a glaring exception (though 2.X was supported for 12 more years after 3.X's release). Sometimes, though, changes initially appear as optional extensions, which are disabled by default. To enable such an extension in Pythons that predate its official arrival, use a special import statement of this form: `python from future import featurename` When coded in a script, this statement must appear as the first executable statement in the file (possibly following a docstring or comment), because it enables special compilation of code on a per-module basis. It's also possible to submit this statement at the interactive prompt to experiment with

th upcoming language changes; the feature will then be available for the remainder of the interactive session. For example, the prior edition of this book used this statement in Python 2.X to activate 3.X true division of Chapter 5, 3.X print calls of Chapter 11, and 3.X absolute imports for packages of Chapter 24. Earlier editions used this statement form to demonstrate generator functions, which require a yield that was not yet enabled by default. This edition is boldly going forward with the present, but future can be used in older Pythons to enable Python 3.7's StopIteration "bubbling" behavior described at the end of Chapter 20: `python from future import generatorstop` For a list of futurisms you may import and turn on this way, see the Python library manual's entry for future. Per its documentation, none of its feature names will ever be removed, so it's safe to leave in a future import even in code run by a version of Python where the feature is enabled normally. The future does not erase the past.

中文翻译

说到变更，可能破坏现有代码的 Python 修改通常都是逐步引入的。这并不总是像它看起来那样“渐进”，3.0 版本就是一个明显的例外（尽管 2.X 在 3.X 发布后又获得 12 年的支持）。不过，有时变更最初是以可选扩展的形式出现，默认是禁用的。要在早于其正式发布日期的 Python 版本中启用这样的扩展，需要使用如下特殊形式的 import 语句：`python from future import featurename` 写在脚本中时，这

条语句必须是文件里第一条可执行语句（可以跟在 `docstring` 或注释之后），因为它启用了基于每个模块的特殊代码编译。你也可以在交互式提示符下提交这条语句来试验即将到来的语言变更；该特性将在本次交互会话的剩余时间内可用。例如，本书上一版曾在 Python 2.X 中使用这条语句来启用：第 5 章的 3.X 真除法（true division）、第 11 章的 3.X `print` 调用，以及第 24 章包的 3.X 绝对导入。更早的版本用这种语句形式演示生成器函数（generator function），它们需要一个默认尚未启用的 `yield`。本版大胆地直奔当下，不过 `future` 可以在更老的 Python 中启用第 20 章末尾描述的 Python 3.7 的 `StopIteration` "冒泡"（bubbling）行为：`python from future import generatorstop` 关于你可以这样导入并开启的"未来主义"（futures）特性列表，参见 Python 库手册中 `future` 的条目。按其文档说明，这些特性名永远不会被移除，所以即使在特性已正常启用的 Python 版本上运行代码，保留 `future` 导入也是安全的。未来不会抹去过去。

深度理解

·核心概念：`future` 是 Python 的"时间机器"——让老版本 Python 提前体验新语法/语义，实现语言变更的平滑过渡。

·底层实现：`future` 是一个真实存在的标准库模块，但它的"真实内容"由编译器硬编码。当解释器解析源码看到 `from future import X` 时（必须在文件头部），它会开启对应的编译标志（compiler flag），让字节码编译阶段采用新语义；在 REPL 中则以 `sys.flags` 级的会话标志生效

。对模块而言，这是"按文件粒度"的编译期开关，所以必须放在最前面。

·设计原因：Python 2→3 转型的惨痛教训。有些破坏性变更没法静默完成，于是先做成可选特性，让大家提前几年试用、迁移，再逐步默认化。generatorstop 就是这类：它改变生成器里 StopIteration 异常的处理语义（第 20 章讲过）。

·实际问题：今天（Python 3.12 时代）几乎所有特性都已落地，`from future import ...` 在业务代码里几乎见不到了，但历史代码、老库兼容层里仍可能出现。作为读者，你要能看懂它。

·初学者误区：以为 future 能"导入未来的功能"（比如导入还没实现的 API）——不是！它只是开启编译器开关。也别把它放在文件中间——会直接 `SyntaxError`。

·生活例子：像市政工程修路先铺"临时便桥"——新路（新特性）没通之前，先让行人（旧代码）从便桥走（future），等新路正式通车，便桥永久保留为历史纪念（特性名永不删除）。

25.7 双用途模式：name 与 main (Dual-Usage Modes: name and main)

英文原文

Our next module-related trick lets you both import a file as a module and run it as a standalone script, a h

ook that is widely used in Python files. It's actually so simple that some learners miss the point at first. Each module has a built-in attribute called `name`, which Python creates and assigns automatically as follows: - If the file is being run as a top-level script file, `name` is set to the string `'main'` when it starts. - If the file is being imported instead, `name` is set to the module's name as known by its clients. The upshot is that a module can test its own name to determine whether it's being run or imported. For example, suppose we create the code file named `dualmode.py` in Example 25-4, with a single function called `title`. Example 25-4. `dualmode.py`

```
python def title(): print('Learning Python, 6E') if name == 'main': # Only when run title() # Not when imported
```

This module defines a function for clients to import and use as usual:

```
python $ python3 >>> import dualmode >>> dualmode.title() Learning Python, 6E
```

But the module also includes code at the bottom that is set up to call the function automatically when this file is run as a program:

```
python $ python3 dualmode.py Learning Python, 6E
```

In effect, a module's `name` variable serves as a usage mode flag, allowing its code to be leveraged as both

an importable library and a top-level script. Though simple, you'll see this hook used in many of the Python program files you are likely to encounter in the wild—both for testing and dual usage. For instance, one of the most common ways you'll see the name test applied is for self-test code. In short, you can package code that tests a module's exports in the module itself by wrapping it in a name test at the bottom of the file. This way, you can use the file in clients by importing it, but also test its logic by running it from the system shell or other launching scheme. Coding self-test code at the bottom of a file under the name test is probably the most common and simplest unit-testing protocol in Python. It's much more convenient than retyping all your tests at the interactive prompt. (Preview: Chapter 36 will discuss other commonly used options for testing Python code—as you'll see, the unittest and doctest standard-library modules provide more advanced testing tools.) In addition, the name trick is also commonly used when you're writing files that can be useful both as command-line utilities and as tool libraries. For instance, suppose you write a file-finder script in Python. You can get more mileage out of your code if you package it in functions, and add a name test in the file to automatically call those functions when the file is run standalone. That way, the script's code becomes reusable in other programs. **NOTE**: What's in a name? Don't confuse the main hook her

e with the main.py file discussed in the prior chapter. That file serves as a script when running an entire package folder as a program, but testing whether name is 'main' is used to give two roles to a single file. Python often conflates the same names for similar but different purposes—see getattr!

中文翻译

下一个与模块相关的技巧让你既能以模块身份导入一个文件，又能把它作为独立的脚本运行——这个钩子在 Python 文件中被广泛使用。它其实简单到有些学习者一开始领会不了重点。每个模块都有一个名为 name 的内置属性，Python 会自动创建并赋值，规则如下：- 如果文件作为顶层脚本文件运行，name 在启动时被设置为字符串'main'。- 如果文件是被导入的，name 则被设置为客户端所知的模块名。结论是：模块可以通过测试自己的 name 来判断自己是"被运行"还是"被导入"。例如，假设我们创建了示例 25-4 中名为 dualmode.py 的代码文件，里面只有一个名为 title 的函数。示例 25-4. dualmode.py

```
python def title(): print('Learning Python, 6E') if name == 'main': # 仅当被运行时title() # 被导入时不执行这个模块定义一个函数供客户端照常导入和使用：python $ python3 >>> import dualmode >>> dualmode.title() Learning Python, 6E
```

但这个模块底部还包含一段代码，当文件作为程序运行时，它会被设置成自动调用该函数：python \$ python3 dualmode.py Learning Python, 6E 实际上，模块的 name 变量充当了用途模式标志（usage mode flag），让它的代码既能作为可导入的库，又能作为顶层脚本来利用。虽然简单，但你

会发现野生世界中的许多 Python 程序文件都在使用这个钩子——既用于测试，也用于双用途。例如，name 测试最常见的应用之一是自测试 (self-test) 代码。简而言之，你可以把测试模块导出物的代码打包进模块自身——放在文件底部的 name 测试里。这样，客户端可以导入它来使用，你也可以从系统 shell 或其他启动方式运行它来测试逻辑。在文件底部的 name 测试下编写自测试代码，大概是 Python 中最常见、最简单的单元测试协议。这比在交互式提示符下重新敲一遍所有测试方便得多。（预告：第 36 章将讨论测试 Python 代码的其他常用选项——如你所见，unit test 和 doctest 标准库模块提供了更高级的测试工具。）此外，name 技巧也常用于编写那些既能当命令行工具、又能当工具库的文件。例如，假设你写了一个 Python 文件查找脚本。如果把代码打包成函数，并在文件里加一个 name 测试，让文件独立运行时自动调用这些函数，你的代码就能获得更多价值。这样一来，脚本的代码就能在其他程序中复用。注：name 里有什么？不要把这里的 main 钩子与上一章讨论的 main.py 文件混淆。那个文件是在把整个包文件夹当作程序运行时充当脚本的；而测试 name 是否为 'main' 是给单个文件赋予两种角色。Python 常常把相同的名字混用于相似但不同的目的——参见 getattr！

深度理解

- 核心概念：name == 'main' 测试——Python 程序文件最著名的惯用法 (idiom)，让同一个文件“一鱼两吃”：被导入时是库，被运行时是程序。
- 底层实现：Python 在启动时用命令行参数构造一个特殊模块对象，命名为 main，把脚本文件作为它的内容执行

——所以顶层脚本的 `name` 恒为 `'main'`。而被 `import` 的文件则以其导入名命名模块对象，`name` 就是模块名。注意 REPL 里的代码也活在 `main` 里，所以 REPL 中 `name` 也是 `'main'`。

·设计原因：程序员常常写完一个库就想“顺手跑跑看效果”。与其再写一个单独的测试驱动文件，不如给脚本一个“主程序开关”。这也是 Python 不同于 Java（必须有 `main` 方法）的设计——Python 没有 `main` 函数的要求，任何顶层代码在被运行时都会执行。

·实际问题：这是“库文件自测”和“CLI 工具”的统一写法。python3 `minmax.py` 直接跑自测；`import minmax` 干净无副作用。所有正经 Python 项目的入口文件（`manage.py`、`main.py`、CLI 工具）都这么写。

·初学者误区：把 `if name == 'main':` 当魔法背下来而不理解；或者误以为 Python 要求必须有 `main()` 函数——并没有，`main()` 只是惯例。还有人在被导入时意外执行了顶层代码（顶层 `print`、顶层调用），导致 `import` 库时“噼里啪啦”输出一堆东西。

·生活例子：同一份简历（文件）有两种用法——投给公司（被 `import` 当库用），自己也复印一份贴在个人主页（被运行当程序）——靠一张“用途标签”（`name`）决定哪种场景下展示哪部分内容。

代码分析 (Example 25-4)

```
def title():  
    print('Learning Python, 6E')  
  
if __name__ == '__main__':    #  
    title()                   #
```

·解释器如何处理：两种执行路径：(1) `python3 dualmode.py`——解释器把 `dualmode.py` 编译后作为 `main` 模块执行，`name` 被赋为 `'main'`，`if` 条件为真，调用 `title()`；(2) `import dualmode`——导入系统按名字创建模块对象 `dualmode`，`name` 是 `'dualmode'`，条件为假，`title` 函数只是被定义、不会被调用。两条路径都会先编译文件、绑定 `title` 名字——区别只在条件判断的结果。

·设计思想：“测试代码与库代码共存但互不干扰”——这是 Python 对“单一文件，双重身份”问题的优雅回答，比 Java/C++ 要求单独入口类灵活得多。

25.8 示例：用 `name` 做单元测试 (Example: Unit Tests with `name`)

英文原文

In fact, we've already seen numerous cases in this book where the name check could be useful. As one example, we coded a script in Chapter 18 that computed the minimum value from the set of arguments sent—Example 18-3, whose code is repeated here for ease of reference:

```
python def minmax(test, args): res = args[0] for arg i
n args[1:]: if test(arg, res): res = arg
return res def lessthan(x, y): return x < y def grtrthan(
x, y): return x > y print(minmax(lessthan, 4, 2, 1, 5, 6,
3)) # Self-test code print(minmax(grtrthan, 4, 2, 1, 5,
6, 3))
```

This script includes self-test code at the bottom, so we can test it without having to retype test code in the REPL each time we run it. The problem with the way it is currently coded, however, is that the output of the self-test call will appear when this file is imported from another file to be used as a tool—not exactly a client-friendly feature! To do better, we can wrap up the self-test call in a name check so that it will be launched only when the file is run as a top-level script, not when it is imported. Example 25-5 lists this new-and-improved version of the module.

Example 25-5. minmax.py

```
python print('I am:', name) def minmax(test, args): re
s = args[0] for arg in args[1:]: if test(arg, res): res = ar
g
return res def lessthan(x, y): return x < y def grtrthan(
x, y): return x > y if name == 'main': print(minmax(les
sthan, 4, 2, 1, 5, 6, 3)) # Self-test code print(minmax(g
rtrthan, 4, 2, 1, 5, 6, 3))
```

We're also printing the value of name at the top here to trace its value (something you wouldn't do in a real library module). Python creates and assigns this usage-mode variable as soon as

it starts loading a file. When we run this file as a top-level script, its name is set to `main`, so its self-test code kicks in automatically:

```
python $ python3 minmax.py I am: main 1 6
```

If we import the file, though, its name is not `main`, so we must explicitly call the function to make it run:

```
python $ python3 >> import minmax I am: minmax
>> minmax.minmax(minmax.lessthan,'hack')'a'
```

Again, regardless of whether this is used for testing, the net effect is that we get to use our code in two different roles—as a library module of tools, or as an executable program. It's buy-one-get-one code. You'll also see programs that route program-mode runs into a module called `main`:

```
python def main(): ... if name == 'main': main()
```

This works, but it's extra code, and there's nothing special about a function named `main` in Python—unlike some other languages, which may be part of the inspiration for this indirection's appearance in Python code.

NOTE: Fishing tutorial past—For another example of the `name == 'main'` test at work, see the `dual-mode script/module formats.py` in this book's examples package. It formats numbers with commas and currency conventions and demos how you can code your own flexible tools instead of relying on built-ins. It didn't a

dd much here and was cut in this edition for space, but provides optional self-study code—and underscores that learning to fish generally beats being given one.

中文翻译

事实上，本书中我们已经见过许多可以用上 name 检查的场景。举一个例子：我们在第 18 章编写过一个脚本，从传入的一组参数中计算最小值——即示例 18-3，其代码在此重复，便于查阅：

```
python def minmax(test, args): res = args[0] for arg in args[1:]: if test(arg, res): res = arg return res def lessthan(x, y): return x < y def grtrthan(x, y): return x > y print(minmax(lessthan, 4, 2, 1, 5, 6, 3)) # 自测试代码print(minmax(grtrthan, 4, 2, 1, 5, 6, 3))
```

这个脚本在底部包含了自测试代码，这样我们不必每次运行都在 REPL 里重新敲测试代码。不过它当前写法的问题是：当这个文件被另一个文件导入用作工具时，自测试调用的输出也会出现——这可不是什么对客户端友好的特性！为了做得更好，我们可以把自测试调用包进一个 name 检查里，使它只在文件作为顶层脚本运行时才启动，导入时不启动。示例 25-5 列出了这个升级版模块。示例 25-5. minmax.py

```
python print('I am:', name) def minmax(test, args): res = args[0] for arg in args[1:]: if test(arg, res): res = arg return res def lessthan(x, y): return x < y def grtrthan(x, y): return x > y if name == 'main': print(minmax(lessthan, 4, 2, 1, 5, 6, 3)) # 自测试代码print(minmax(grtrthan, 4, 2, 1, 5, 6, 3))
```

我们还在顶部打印 name 的值来追踪它（真实库模块里你不会这么做）。Python 在开始加

载文件的那一刻就创建并赋值了这个用途模式变量。当我们作为顶层脚本运行这个文件时，它的名字被设为 `main`，于是自测试代码自动启动：`python $ python3 minmax.py I am: main 1 6` 但如果我们导入这个文件，它的名字就不是 `main` 了，所以必须显式调用函数才能让它运行：`python $ python3 >>> import minmax I am: minmax >>> minmax.minmax(minmax.lessthan,'hack')'a'`再说一遍，无论这是否用于测试，净效果都是：我们的代码可以扮演两种角色——作为工具库模块，或者作为可执行程序。这是“买一送一”的代码。你还会看到一些程序把“程序模式”的运行路由到一个名为 `main` 的模块：`python def main(): ... if name == 'main': main()` 这样也行，但这是多余的代码，而且名为 `main` 的函数在 Python 中没有任何特殊之处——不像某些其他语言那样（那些语言可能正是这种间接写法出现在 Python 代码中的灵感来源）。注：钓鱼教程往事——关于 `name == 'main'` 测试的另一个实例，参见本书示例包中双模式脚本/模块 `formats.py`。它用逗号和货币惯例格式化数字，演示了如何编写自己的灵活工具而不是依赖内置功能。它在这里贡献不多，本版因篇幅被删减，但提供了可选的自我学习代码——也强调了一个道理：学会钓鱼（learning to fish）通常胜过别人直接给你一条鱼。

深度理解

- 核心概念：把“自测代码”从“无条件顶层执行”改成“`name` 守卫下执行”，模块的导入副作用（import side effects）瞬间归零——这是本节的本质改进。
- 底层实现：`import minmax` 时，模块执行顺序是：打印 `I am: minmax` → 定义三个函数 → 判断 `name == 'main'`

' 为假 → 跳过自测。而 python3 minmax.py 时, name 是'main', 自测两行 print 依次执行。'hack' 是序列解包参数 (把字符串展开成单个字符), minmax(lessthan,'h','a','c','k') 逐个比较求出最小字符'a'。

·设计原因: "库文件可自测"把测试成本降到最低——不需要单独建 test 文件, 不需要测试框架, 一个文件搞定。这符合 Python"简单优先"的哲学; 更专业的 unittest/doctest 是第 36 章的事。

·实际问题: 写任何工具脚本时, "顶层放 if name == 'main': 调 main()"是标准模板。它也解决了"想复用一个脚本"的问题: 脚本的每个功能都写成函数, 运行入口只是最底下几行。

·初学者误区: ① 忘了加守卫, 库被导入时疯狂打印测试输出; ② 以为 main 是关键字、必须有——Python 里 def main() 只是个普通函数名; ③ 在 if name == 'main': 里定义函数——那这些函数就只在脚本模式存在, 导入模式用不了。

·生活例子: 自测代码像新家具附带的"组装说明书"——你买回家 (import) 时说明书安静地躺在箱底 (不执行); 但你想在店里试装展示 (python3 运行) 时, 店员会照着说明书现场组装一遍 (自测执行)。

代码分析 (Example 25-5)

```

print('I am:', __name__)                                # __na...

def minmax(test, *args):
    res = args[0]                                        #
    for arg in args[1:]:                                #
        if test(arg, res):                              #
            res = arg                                    # ""
    return res

def lessthan(x, y): return x < y                        #
def grtrthan(x, y): return x > y                        #

if __name__ == '__main__':
    print(minmax(lessthan, 4, 2, 1, 5, 6, 3))           #
    print(minmax(grtrthan, 4, 2, 1, 5, 6, 3))

```

·解释器如何处理：minmax 是"策略模式"的雏形——比较逻辑（test）作为参数传入，函数本身与具体比较方式解耦。运行时：minmax(lessthan, 4, 2, 1, 5, 6, 3) 把 test 绑定到 lessthan，遍历剩余参数维护最小值，返回 1；第二个调用用 grtrthan 求最大值，返回 6。参数 args 在调用处被收集成元组 (4, 2, 1, 5, 6, 3)，args[1:] 切片跳过第一个元素。

·设计思想：把"算法骨架"与"具体策略"分离——同样的遍历逻辑，传入不同比较函数就得到不同结果。这是函数式思维在 Python 中的经典体现，也为下一部分类的多态打下伏笔。

25.9 import 与 from 的 as 扩展 (The as Extension for import and from)

英文原文

Next on the tour is a follow-up on a topic introduced in Chapter 23's name-collision coverage. As a minor but useful convenience, both the `import` and `from` statements support an optional `as` clause, which simply renames a name imported by your script. For example, the following `import` statement using the `as` extension:

```
python import modulename as name # And use name, not modulename is equivalent to the following set of three statements, which renames the module in the importer's scope only (it's still known by its original name to other files), and drops the original name in the importer's scope altogether:
```

```
python import modulename # Run a normal import
name = modulename # Rename the module - here del
modulename # Discard the original name - here
```

After an `import` with `as`, you can—and in fact, must—use the name listed after the `as` to refer to the module. The longer equivalent works because modules are first-class objects just like functions, and can be passed around freely. The `as` extension works in a `from` statement, too, to assign a name imported from a file to a different name in the importer's scope. As before, you get only the new name you provide, not its original:

python from modulename import attrname as name
And use name, not attrname This in turn works the same as the following statements:

```
python from modulename import attrname name = attrname del attrname
```

As noted in Chapter 23, this extension is commonly used both to provide shorter synonyms for longer names and to avoid name clashes when you are already using a name in your script that would otherwise be overwritten by a normal import:

```
python import reallylongmodulename as name # Use shorter nickname name.func() # Rename to make shorter
```

```
from module1 import utility as util1 # Can have only one "utility" from module2 import utility as util2 # Rename to make unique util1(); util2()
```

By way of review, the `as` clause also comes in handy for providing a short, simple name for an entire directory path and avoiding name collisions when using the package import feature described in Chapter 24:

```
python import dir1.dir2.mod as mod # Only list full path once mod.func() # Only one change if path changes
```

```
from dir1.dir2.mod import func as modfunc # Rename to make unique if needed modfunc() # Allow func to be something else
```

Finally, the `as` clause is also something of a hedge against name changes: if a new release of a library renames a module or tool your code uses extensively, or provides a new alternative you'd rather use instead, you can simply rename it to its prior name on import to avoid breaking your code:

```
python import newname as oldname from library import newname as oldname ... and keep happily using oldname until you have time to update all your code
```

... That said, if all software changes were just name changes, we'd have a lot less to fill our time!

中文翻译

接下来要讲的是第 23 章“名字冲突”部分引入话题的后续。作为一个虽小但很有用的便利特性，`import` 和 `from` 语句都支持可选的 `as` 子句，它只是重命名你的脚本导入的名字。例如，下面这条使用 `as` 扩展的 `import` 语句：`python import modulename as name` # 用 `name`，而不是 `modulename` 等价于下面这组三条语句——它只在导入方的作用域内重命名模块（对别的文件来说它仍叫原名），并彻底丢弃导入方作用域里的原名：`python import modulename` # 执行普通导入 `name = modulename` # 在这里重命名模块 `del modulename` # 在这里丢弃原名在带 `as` 的 `import` 之后，你可以——事实上是必须——用 `as` 后面列出的名字来引用这个模块。这个更长的等价写法之所以可行，是因为模块和函数一样都是头等对象（first-class objects），可以自由传递。`as` 扩展同样适用于 `from` 语句，可以把从文件中导入的名字赋给导入方作用域里的另一个名字。

和之前一样，你只得到你提供的新名字，而不是原名：`python from modulename import attrname as name` # 用 `name`，而不是 `attrname` 这又等同于下面的语句：`python from modulename import attrname name = attrname del attrname` 如第 23 章所述，这个扩展常被用来：为较长的名字提供更短的别名（shorter synonyms），以及在你的脚本里已经用了一个名字、而普通导入会覆盖它时避免名字冲突（name clashes）：`python import reallongmodulename as name` # 用更短的昵称 `name.func()` # 重命名以缩短 `from module1 import utility as util1` # 只能有一个 "utility" `from module2 import utility as util2` # 重命名使其唯一 `util1(); util2()` 顺便复习一下：`as` 子句在为整个目录路径提供简短易记的名字、以及使用第 24 章描述的包导入（package import）特性时避免名字冲突方面也很方便：`python import dir1.dir2.mod as mod` # 完整路径只需写一次 `mod.func()` # 路径变了只改一处 `from dir1.dir2.mod import func as modfunc` # 需要时重命名使其唯一 `modfunc()` # 让 `func` 这个名字留作他用最后，`as` 子句还算是应对名字变更的一种对冲手段：如果库的新版本重命名了你代码中大量使用的模块或工具，或者提供了你更想用的新替代品，你只需在导入时把它重命名为旧名字，就能避免破坏你的代码：`python import newname as oldname from library import newname as oldname` ...然后愉快地继续用 `oldname`，直到你有时间更新所有代码... 话说回来，如果所有软件变更都只是改个名字，我们的工作可就要清闲太多了！

深度理解

·核心概念：`as` 子句 = 导入时重命名。本质是"导入 + 赋值 + 删除原名"三步的语法糖。模块是头等对象，所以能被任意赋值给新名字。

·底层实现：`import module_name as name` 的字节码序列相当于：先执行普通导入（`IMPORTNAME`），把模块对象压栈，再执行 `STOREFAST name`（绑定到 `name`），原名的绑定被跳过——所以导入方作用域里只有 `name` 一个名字。

·设计原因：长模块名（`from sqlalchemy.orm.session import SessionMaker`）影响可读性；两个模块导出的工具重名（比如两个库都有 `util`）就需要改名消歧。`as` 让"改名"成为导入的一部分，不必写三行代码。

·实际问题：`import numpy as np`、`import pandas as pd` 就是最著名的应用。包路径 `import dir1.dir2.mod as mod` 让你只写一次长路径；库升级改名时用 `as` 保住旧名字，是"零成本兼容层"。

·初学者误区：以为 `import a as b` 之后 `a` 还能用——不能！绑定后只有 `b`。以为 `as` 改变了模块在 `sys.modules` 里的注册名——没有，注册名永远是模块真名，`as` 只影响导入方局部命名空间。

·生活例子：`as` 像给外国同事起中文昵称——文件里他的护照名（模块真名）不变，但你在办公室里（导入方作用域）叫他"老王"就行，遇到两个都叫"John"的人（重名），就一个叫"大 John"一个叫"小 John"（`as` 消歧）。

代码分析

```
import reallylongmodulename as name      #
name.func()                              #

from module1 import utility as util1      # "utility"
from module2 import utility as util2      #
util1(); util2()
```

·解释器如何处理：第一段：解释器按 `reallylongmodulename` 完成查找、加载、注册到 `sys.modules`，然后把模块对象绑定给局部名字 `name`——之后 `name.func()` 与 `reallylongmodulename.func()` 指向同一个函数对象。第二段：两次 `from ... as` 分别把两个不同模块里的 `utility` 函数绑定到 `util1`、`util2`——作用域里同时存在两个互不干扰的名字，各自指向自己的函数对象。注意这种“复制”与第 23 章讲的 `from` 语义一致：绑定的是对象引用，不是别名链接。

·设计思想：`as` 把“命名”从导入点分离出来，让名字管理（缩短、消歧、兼容）成为导入语句的一部分——一处声明，处处干净。

25.10 模块内省 (Module Introspection)

英文原文

Next up is module plumbing. Because modules expose most of their interesting properties as built-in attributes, it's easy to write programs that manage other programs—tools we usually call metaprograms, beca

use their subjects are other programs. This domain is also referred to as introspection, because programs can see and process object internals. Introspection is a somewhat advanced feature, but it can be useful for building programming tools.

For instance, to fetch the value of a module's attribute, we can use attribute qualification or index the module's attribute dictionary, exposed in the built-in `dict` attribute we explored in Chapter 23. As we've also seen, Python's `vars` built-in is an alternative way to access `dict`, and its `sys.modules` dictionary records all loaded modules by import-name string. In addition, its `getattr` built-in lets us fetch attributes from their string names—it's like saying `object.attr`, but `attr` is an expression that produces a string at runtime. Hence, all the following expressions reach the same attribute and object named `name` after importing `M` and `sys`:

```
python M.name # Qualify object by attribute
M.dict['name'] # Index namespace dictionary manually
vars(M)['name'] # Namespace dictionary alternative
sys.modules['M'].name # Index loaded-modules table manually
getattr(M,'name') # Call built-in fetch function
sys.modules['M'].dict['name'] # Module and attribute name strings
```

Demoing with Example 25-5 (and chained comparisons that imply an `and` and a right-side repeat):

```
python >> import minmax, sys >> (minmax.lessthan
... is minmax.dict['lessthan'] is vars(minmax)['lessthan']
... is sys.modules['minmax'].lessthan is getattr(minmax, 'lessthan')
... is sys.modules['minmax'].dict['lessthan']) True
```

Of course, the first of these is much easier on the eyes (and keyboard), but the others support more generic access.

中文翻译

接下来是模块的“管道工程”。因为模块把大部分有趣的属性都以内置属性（built-in attributes）的形式暴露出来，所以很容易编写管理其他程序的程序——这类工具我们通常称为元程序（metaprograms），因为它们的处理对象是其他程序。这个领域也被称为内省（introspection），因为程序能够查看并处理对象的内部。内省是个有些高级的特性，但对构建编程工具很有用。例如，要获取模块属性的值，我们可以用属性限定（attribute qualification），也可以索引模块的属性字典——它通过第 23 章探索过的内置属性 dict 暴露出来。如我们所见，Python 的 vars 内置函数是访问 dict 的另一种方式，而 sys.modules 字典按导入名字字符串记录所有已加载的模块。此外，getattr 内置函数让我们能用字符串名字获取属性——它就像 object.attr，只不过 attr 是运行时产生字符串的表达式。因此，在导入 M 和 sys 之后，下面所有表达式都能到达同一个名为 name 的属性和对象：

```
python M.name # 用属性限定对象M.dict['name'] # 手动索引命名空间字典vars(M)['name'] # 命名空间字典的替代写法sys.modules['M'].name # 手动索
```

引已加载模块表`getattr(M, 'name')` # 调用内置获取函数`sys.modules['M'].dict['name']` # 模块名和属性名都是字符串
用示例 25-5 演示（链式比较隐含着 `and` 和右侧的重复求值）：
`python >>> import minmax, sys >>> (minmax.lessthan ... is minmax.dict['lessthan'] is vars(minmax)['lessthan'] ... is sys.modules['minmax'].lessthan is getattr(minmax, 'lessthan') ... is sys.modules['minmax'].dict['lessthan'])` True
当然，第一种写法对眼睛（和键盘）友好得多，但其他写法支持更通用的访问方式。

深度理解

·核心概念：内省 = 程序反过来检查程序。模块的一切（属性、名字、文件位置、加载状态）都可通过标准 API 在运行时读取，这使 Python 成为编写“程序员工具”（linter、文档生成器、调试器、热重载）的理想语言。

·底层实现：六种访问路径殊途同归：(1) `M.name` 编译为属性访问指令，运行时在模块 `dict` 里按名字找；(2) `M.dict['name']` 直接索引字典；(3) `vars(M)` 等价于 `M.dict`；(4) `sys.modules['M']` 从已加载模块注册表取出模块对象；(5) `getattr(M, 'name')` 是“字符串 → 属性”的动态获取；(6) 组合拳 `sys.modules['M'].dict['name']` 纯字符串路径。链式 `is` 比较等价于 `a is b and b is c and ...`，全真输出 True，证明六个路径拿到的是同一个函数对象（is 比较对象身份）。

·设计原因：动态语言的一大红利——名字就是字典键，属性就是可遍历的字典项。静态语言要反射（reflection）框架才做得到的事，Python 原生就有。

·实际问题：dir()、help()、PyDoc、调试器、pytest 的收集机制、Django 的自动注册，全部建立在"能看模块内部"之上。

·初学者误区：以为 getattr 是"魔法"——它就是 M.dict[name] 的容错版（还会处理 getattr 钩子）。以为 vars(M) 返回副本——它返回的是真实字典，修改它会改到模块！

·生活例子：内省像"安检扫描仪"——不用打开行李箱（对象），扫描仪（dict、getattr）就能看清里面每件物品（属性）的名字和价值。

代码分析

```
>>> import minmax, sys
>>> (minmax.lessthan
...     is minmax.__dict__['lessthan'] is vars(minmax)['lessthan']
...     is sys.modules['minmax'].lessthan is getattr(minmax, 'les...
...     is sys.modules['minmax'].__dict__['lessthan'])
True
```

·解释器如何处理：六个子表达式逐一求值。minmax.lessthan 走 LOADATTR 指令；minmax.dict['lessthan'] 走"取字典再 BINARYSUBSCR（下标取键）"；vars(minmax) 是内置函数调用返回模块字典；sys.modules['minmax'] 从注册表取回同一个模块对象；getattr(minmax, 'lessthan') 是 C 实现的属性查找。链式 is 逐个比较对象身份（比较内存地址），全部相等才返回 True。它们拿到的是同一个函数对象，只是"寻路方式"不同。

·设计思想：统一寻址——同一个东西给六条路，读写数据

时用最直观的 `M.name`，写通用工具（元编程）时用字符串路径。这正是“简单用法友好、高级用法强大”的体现。

25.11 示例：用 dict 列出模块 (Example: Listing Modules with dict)

英文原文

By exposing module internals like this, Python helps you build programs about programs. As a demo, the module in Example 25-6, named `mydir.py`, puts these ideas to work to implement a customized and expanded version of the built-in `dir` function. It defines and exports a function called `listing`, which takes a module object as an argument and prints a formatted display of the module's namespace sorted by attribute name.

Example 25-6. `mydir.py`

python `"""mydir.py: a module that lists the namespaces of other modules. Import this module's listing and pass an imported module, or run this file as a script to perform its self-test code.`

`"""`

`sepchr = '-' seplen = 60 def listing(module, verbose=True, unders=True): """List module: just attributes if verbose=False, hide built-in X attributes if unders=False.`


```
"""sepline = sepchr seplen
```

```
if verbose: print(sepline) print(f'name: {module.name}
\file: {module.file}') print(sepline) # Scan namespace
keys for (count, attr) in enumerate(sorted(module.dic
t)): prefix = f'{count + 1:02d} {attr}' if attr.startswith("
): if unders: print(prefix, '<built-in name>') # Skip file,
etc. else: print(prefix, getattr(module, attr)) # Or mod
ule.dict[attr] if verbose: print(sepline) print(f'{module.
name} has {count + 1} names') print(sepline) if name
=='main': import mydir listing(mydir) # Self-test code
: list myself
```

Notice the docstrings in this module; because we may want to use this as a general tool, the docstrings provide functional information accessible via help and the browser mode of PyDoc—tools that use similar introspection tools to do their jobs (see Chapter 15 for usage info). A self-test is also provided at the bottom of this module, which narcissistically imports and lists itself; here's the sort of output produced (with path edits for space):

```
python $ python3 mydir.py-----
-----name: mydir file: /Users/
me/.../LP6E/Chapter25/mydir.py -----
----- 01) builtins <built-i
n name> 02) cached <built-in name> 03) doc <built-
in name> 04) file <built-in name> 05) loader <built-i
n name> 06) name <built-in name> 07) package <bu
```

```

ilt-in name> 08) spec <built-in name> 09) listing <fu
nction listing at 0x1077758a0> 10) sepchr- 11) seple
n 60 -----
-----mydir has 11 names -----
-----

```

To use this as a tool for listing other modules, simply pass the modules in as objects to this file's function. Here it is listing itself manually, as well as attributes in the tkinter GUI module in the Python standard library; it will technically work on any object with name, file, and dict attributes:

```

python >> from mydir import listing >> import mydir, tkinter >> listing(mydir, unders=False, verbose=False) 09) listing <function listing at 0x10800fba0> 10) sepchr - 11) seplen 60 >> listing(tkinter, unders=False) -----
--name: tkinter file: /.../lib/python3.12/tkinter/init.py
-----
-- 01) ACTIVE active 02) ALL all 03) ANCHOR anchor
04) ARC arc
... more names omitted
... 166) re <module're' from'/.../lib/python3.12/re/init.py'> 167) sys <module'sys' (built-in)> 168) types <module'types' from'/.../lib/python3.12/types.py'> 169) wantobjects 1 -----
-----tkinter has 169 names

```

You'll meet `getattr` and its relatives again later. The point to notice here is that `mydir` is a program that lets you browse other programs. Because Python exposes its internals, you can process objects generically.

NOTE: REPL startup tip—You can preload tools such as `mydir.listing` and the reloader we'll code in a moment into the interactive REPL by importing them in a file named by the `PYTHONSTARTUP` environment variable. Because code in the startup file runs in the interactive namespace, importing common tools in this file can save you some typing. See Appendix A for more info.

中文翻译

通过这样暴露模块内部，Python 帮助你构建“关于程序的程序”。作为演示，示例 25-6 中名为 `mydir.py` 的模块把这些思想付诸实践，实现了一个定制化、扩展版的 `dir` 内置函数。它定义并导出一个名为 `listing` 的函数，该函数接受一个模块对象作为参数，按属性名排序打印模块命名空间的格式化展示。示例 25-6. `mydir.py`

```
python """ mydir.py: 列出其他模块命名空间的模块。导入本模块的 listing 并传入一个已导入的模块，或者把本文件作为脚本运行来执行其自测试代码。"""
sepchr = '-'
seplen = 60
def listing(module, verbose=True, unders=True):
    """列出模块：若 verbose=False 只列属性，若 unders=False 隐藏内置 X 属性。"""
    sepline = sepchr * seplen
    if verbose:
        print(sepline)
    print(f'name: {module.name}\nfile: {module.file}')
    print(sepline)
    # 扫描命名空间键
    for (count, attr) in enumer
```

```

ate(sorted(module.dict)): prefix = f'{count + 1:02d}) {
attr}'if attr.startswith('): if unders: print(prefix,'<built-
in name>') # 跳过 file 等else: print(prefix, getattr(mod
ule, attr)) # 或 module.dict[attr] if verbose: print(sepli
ne) print(f'{module.name} has {count + 1} names') pri
nt(sepline) if name == 'main': import mydir listing(my
dir) # 自测试代码：列出我自己注意这个模块里的文档字符
串 (docstrings) ; 因为我们可能把它当作通用工具使用，
文档字符串通过 help 和 PyDoc 的浏览器模式提供了功能
信息——这些工具本身就用类似的自省手段完成工作（用
法见第 15 章）。这个模块底部还提供了自测试，它自恋地
导入并列出自己；下面是它产生的输出样例（路径为节省篇幅
已略改）：python $ python3 mydir.py-----
-----name: mydir
file: /Users/me/.../LP6E/Chapter25/mydir.py-----
-----01) builti
ns <built-in name> 02) cached <built-in name> 03)
doc <built-in name> 04) file <built-in name> 05) loa
der <built-in name> 06) name <built-in name> 07) p
ackage <built-in name> 08) spec <built-in name> 09
) listing <function listing at 0x1077758a0> 10) sepchr
-11) seplen 60-----
-----mydir has 11 names-----
-----要把它当作列出其
他模块的工具使用，只需把模块作为对象传给这个文件的函
数。下面它手动列出自己，以及标准库中 tkinter 这个 GUI
模块的属性；技术上，它对任何具有 name、file、dict 属
性的对象都有效：python >>> from mydir import listin

```

```

g >>> import mydir, tkinter >>> listing(mydir, under
s=False, verbose=False) 09) listing <function listing a
t 0x10800fba0> 10) sepchr -11) seplen 60 >>> listing
(tkinter, unders=False)-----
-----name: tkinter file: /.../lib/pyth
on3.12/tkinter/init.py-----
-----01) ACTIVE active 02) ALL all 0
3) ANCHOR anchor 04) ARC arc ...更多名字已省略... 16
6) re <module're' from'/.../lib/python3.12/re/init.py'>
167) sys <module'sys' (built-in)> 168) types <module
'types' from'/.../lib/python3.12/types.py'> 169) wanto
bjects 1-----
-----tkinter has 169 names 你以后还会再次遇到 ge
tattr 和它的近亲们。这里要注意的重点是：mydir 是一个
让你浏览其他程序的程序。因为 Python 暴露了内部结构，
你可以通用地处理对象（process objects generically）。
注：REPL 启动技巧——你可以把 mydir.listing 这样的工
具，以及我们一会儿要写的重载器，预加载到交互式 REPL
中：把它们 import 进一个由 PYTHONSTARTUP 环境变
量命名的文件即可。因为启动文件中的代码在交互命名空间
里运行，在这个文件里导入常用工具能省去你不少敲键盘的
功夫。更多信息见附录 A。

```

深度理解

·核心概念：mydir.py 是一个“元程序”样板——不关心列出的模块是谁，只要它有 name、file、dict，就能格式化输出。通用性来自对字典的遍历，而非对特定模块的硬编码。

·底层实现：`module.dict` 是普通 `dict`，`sorted()` 对键排序保证输出有序；`enumerate()` 生成序号；`attr.startswith('')` 判断是否是 X 内置名字；`getattr(module, attr)` 用字符串取属性值。`{count + 1:02d}` 是格式化：两位数补零。自测试里 `import mydir; listing(mydir)`——“自己导入自己”，因为脚本模式下 `mydir` 这个名字还没被绑定（脚本名是 `main`）。

·设计原因：内置 `dir()` 输出太朴素（一堆名字挤在一起），且不区分“内置元数据”与“用户定义名字”。`listing` 用可读的编号、分类、分隔线重排信息——工具的价值在于“把原始数据变成信息”。

·实际问题：类似的思路遍布生态——`pydoc`、`inspect` 模块、`IPython` 的 `whos`、各种“模块浏览器”。`PYTHONSTARTUP` 让 REPL 一打开就有自定义工具可用，是重度 REPL 用户的效率法宝。

·初学者误区：不知道每个模块其实有 8 个左右的内置名字（`builtins`、`cached`、`doc`、`file`、`loader`、`name`、`package`、`spec`）——第一次列出模块时“咦怎么多了这么多”。也误以为 `dir()` 的输出就是全部——其实 `dir()` 是 `sorted(module.dict.keys())` 的变体（外加继承查找，下一部分讲类时展开）。

·生活例子：`listing` 像公司内部的“员工名录自动生成器”——不管哪个部门（模块），只要有姓名卡（`dict`）就能排版打印；`IDLE/PyDoc` 则像门口的“访客登记系统”，同样在读取员工卡信息。

代码分析 (Example 25-6)

```
def listing(module, verbose=True, unders=True):
    sepline = sepchr * seplen                                # '-' * 60 ...
    if verbose:
        print(sepline)
        print(f'name: {module.__name__}\nfile: {module.__file__}')
        print(sepline)

    for (count, attr) in enumerate(sorted(module.__dict__)): # ...
        prefix = f'{count + 1:02d}) {attr}'
        if attr.startswith('__'):                             #
            if unders:
                print(prefix, '<built-in name>')              #
            else:
                print(prefix, getattr(module, attr))          #
        if verbose:
            print(sepline)
            print(f'{module.__name__} has {count + 1} names')
            print(sepline)
```

·解释器如何处理：'--' 60 执行字符串重复生成 60 个连字符的分隔线。sorted(module.dict) 对字典键排序返回列表，enumerate 逐对产出 (序号, 名字)，count 从 0 计数、显示时 +1 且 :02d 补零成两位数 (01), 09))。startswith('__') 判断内置名字，getattr(module, attr) 在 unders=False 模式下获取其值——对 tkinter 这类大模块，结果多达 169 项，证明标准库模块的 dict 里塞满了各种常量、函数、子模块。

·设计思想：参数化输出——verbose、unders 两个布尔参数控制输出粒度，同一个函数适配“快速浏览”与“详细查看”两种需求。自测代码 import mydir; listing(mydir) 顺

手验证工具自身的正确性——元程序"自举"。

25.12 按名字字符串导入 (Importing Modules by Name String)

英文原文

Finally, it's time for something more dynamic. By now, you've probably noticed that the module name in an `import` or `from` statement is a hardcoded variable name. Sometimes, though, your program will get the name of a module to be imported as a string at runtime—from a user selection in a GUI, or a parse of an XML document, for instance.

Unfortunately, you can't use `import` statements directly to load a module given its name as a string—Python expects a variable name that's taken literally and not evaluated, not a string or expression. For instance:

```
python >> import 'string' SyntaxError: invalid syntax
```

It also won't work to simply assign the string to a variable name:

```
python >> x = 'string' >> import x ModuleNotFoundError: No module named 'x'
```

Here, Python will try to import a file `x.py`, not the string module—the name in an `import` statement both becomes a variable assigned to the loaded module and

d identifies the external file literally.

Running Code Strings To get around this, you need special tools to load a module dynamically from a string that is generated at runtime. The most general approach is to construct an import statement as a string of Python code and pass it to the `exec` built-in function to run:

```
python >> modname = 'string' >> exec('import' + modname) # Run a string of code >> string # Imported in this namespace <module'string' from '/.../lib/python3.12/string.py'>
```

We met the `exec` function—and its cousin for expressions, `eval`—earlier, in Chapters 3, 5, 9, and 10. `exec` compiles a string of code and passes it to the Python interpreter to be executed. In Python, the bytecode compiler is available at runtime, so you can write programs that construct and run other programs like this.

By default, `exec` runs the code in the current scope (as if pasted there), but you can get more specific by passing in optional namespace dictionaries. It also has security issues noted earlier in this part of the book, which may be moot in a code string you are building yourself.

Direct Calls: Two Options The only real drawback to `exec` here is that it must compile the import statement each time it runs, and compiling can be slow. Precompiling to bytecode with the `compile` built-in may help

for code strings run many times, but in most cases, it's probably simpler and may run quicker to use the built-in import function to import from a name string.

The effect is similar, but import returns the module object—assign it to a name to keep it:

```
python >> modname = 'string' >> string = import(modname) >> string <module'string' from '/.../lib/python3.12/string.py'>
```

Because all imports work by invoking import, it loads the named module normally. The newer standard-library call `importlib.importmodule` does the same job; Python's docs describe it as a simplified wrapper around import for "everyday" use (though our code is growing longer as our tools are growing newer):

```
python >> import importlib >> modname = 'string' >> string = importlib.importmodule(modname) >> string <module'string' from '/.../lib/python3.12/string.py'>
```

This call works the same as import in its basic roles, but see Python's manuals for more details on both calls' advanced usage and arguments. Python's docs also seem to prefer the newer `importlib` call for importing by name string, though this seems subjective, either call works, and the import system has been a frequent morpher.

One callout here: both calls also work for the package

e imports of the prior chapter, but the first returns the leftmost component in a package path, and the second returns the last—in fact, this is their most prominent difference:

```
python >> import importlib >> import('email.message') <module'email' from '/.../lib/python3.12/email/init.py'> >> importlib.importmodule('email.message') <module'email.message' from '/.../lib/python3.12/email/message.py'>
```

The `importlib` call also works for a package-relative import string (with leading dots), if also passed the string name of a package path from which to resolve the import. See Chapter 24 for the story on package imports.

中文翻译

终于到了更“动态”的部分。到目前为止你应该已经注意到：`import` 或 `from` 语句中的模块名是一个硬编码的变量名。然而有时，你的程序会在运行时以字符串形式拿到要导入的模块名——比如来自 GUI 中用户的选择，或者对某个 XML 文档的解析结果。不幸的是，你没法直接用 `import` 语句加载一个“名字是字符串”的模块——Python 期望的是一个按字面取、不求值的变量名，而不是字符串或表达式。例如：
`python >>> import 'string' SyntaxError: invalid syntax`
仅仅把字符串赋给一个变量也不管用：`python >>> x = 'string'>>> import x ModuleNotFoundError: No module named 'x'` 这里 Python 会尝试导入名为 `x.py` 的文件，而不是 `string` 模块——因为 `import` 语句中的名字，一方面

会成为赋值给已加载模块的变量，另一方面也按字面标识外部的实际文件。运行代码字符串（Running Code Strings）要绕过这个限制，你需要特殊工具，从运行时生成的字符串动态加载模块。最通用（最直接）的做法是：把一条 `import` 语句构造成 Python 代码字符串，然后交给内置函数 `exec` 执行：

```
python >>> modname = 'string' >>> exec  
( 'import' + modname ) # 运行一段代码字符串 >>> string  
# 在这个命名空间里已被导入 <module 'string' from '/...  
/lib/python3.12/string.py'> 我们之前在第三、五、九、十章就接触过 exec 函数——以及它在表达式上的堂兄弟 eval。exec 会编译一段代码字符串并交给 Python 解释器执行。在 Python 中，字节码编译器在运行时也是可用的，所以你能像这样编写程序来构造并运行其他程序。默认情况下 exec 在当前作用域运行代码（就好像那段代码被粘贴在这里一样），不过你也可以传入可选的命名空间字典，让执行更精确。它还有一些本书早先说过的安全问题，但如果代码字符串是你自己构造的，这些隐患通常可以忽略。直接调用：
```

两个选项（Direct Calls: Two Options）这里 `exec` 唯一真正的缺点是：它每次运行都必须重新编译 `import` 语句，而编译可能较慢。对于要多次运行的代码字符串，可以先用内置函数 `compile` 预编译成字节码，但多数情况下，用内置函数 `import` 按名字字符串导入可能更简单、也可能更快。效果类似的，不过 `import` 返回模块对象——把它赋值给一个名字就能留住它：

```
python >>> modname = 'string'  
>>> string = import(modname) >>> string <module  
'string' from '/.../lib/python3.12/string.py'> 因为所有导入最终都是调用 import 完成的，所以它能以正常方式加载指定模块。较新的标准库调用 importlib.importmodule
```

干的是同样的活；Python 文档把它描述为 `import` 的简化包装，适合“日常”使用（虽然随着工具变新，我们的代码反而越来越长了）：`python >>> import importlib >>> modname = 'string' >>> string = importlib.importmodule(modname) >>> string <module'string' from '/.../lib/python3.12/string.py'>` 在基本角色上这个调用与 `import` 完全一致，但关于两个调用的高级用法与参数，请查阅 Python 文档。Python 文档似乎也更偏爱更“新”的 `importlib` 调用来按名字字符串导入，虽然这好像有点主观——两个都能用，而 `import` 系统本来就是常常变化的东西。这里要补充一点：两个调用同样适用于上一章的包导入，不过第一个返回包路径中最左的组件，第二个返回最末的——这其实是它们最显著的区别：`python >>> import importlib >>> import('email.message') <module'email' from '/.../lib/python3.12/email/init.py'> >>> importlib.importmodule('email.message') <module'email.message' from '/.../lib/python3.12/email/message.py'>` `importlib` 调用还支持包相对导入的字符串（带前导点），只要同时传入用于解析导入的包路径字符串即可。完整故事见第 24 章的包导入。

深度理解

·核心概念：动态导入（dynamic import）——当“模块名”在写代码时还不知道、要到运行时才以字符串形式出现时，如何加载模块。`import` 语句只能接受字面标识符，所以需要 `exec`、`import`、`importlib.importmodule` 三把“钥匙”。这是插件系统、测试框架、配置驱动加载的基础。

·底层实现：import 语句最终会被编译成 `IMPORTNAME` 字节码，运行时调用内置的 `import` 函数完成“查找→加载→注册到 `sys.modules`→返回模块对象”。`exec('import' + mod)` 是先把字符串拼成源码再交给编译器；`importlib.importmodule` 则是官方推荐的 `import` 的友好封装——两者对包路径的返回值不同：`import('email.message')` 返回最左的 `email` 包，`importmodule` 返回最后的 `email.message` 子模块。

·设计原因：`import` 是 Python 的语句，不是函数——语句里的名字被编译器静态解析，不可能“求值一个表达式”。要给程序留一条运行时导入的路，语言设计者就必须额外提供这组 API。`exec/compile` 是通用逃生舱，`import` 是原生的钩子，`importlib` 是面向“正常人”的封装。

·实际问题：插件系统（配置里写模块名，启动时按名加载）、测试 runner、脚本引擎、按需（懒）加载重型模块、跨格式导入器。Python 3.12 中 `importlib` 是官方建议的手段。

·初学者误区：① 以为 `import x` 里的 `x` 会被“当作变量求值”——不会，它是字面名字（所以 `import 'string'` 是语法错误）；② 以为 `import` 与 `importlib.importmodule` 完全等价——在包路径上，一个返回最左、一个返回最右；③ 把不可信的运行时输入直接拼进 `exec/eval`——这是代码注入漏洞，永远先校验。

代码分析

```
>>> modname = 'string'
>>> exec('import ' + modname)          #
>>> string                             #
<module 'string' from '/.../lib/python3.12/string.py'>
```

·解释器如何处理：第一种方式——`exec` 拿到字符串 `'import string'` 后，先交给编译器生成字节码，再在当前作用域执行该字节码，效果等价于在当前位置打字 `import string`：加载、注册 `sys.modules`、把模块对象绑定到名字 `string`。第二种方式——`import('string')` 返回模块对象，赋值给 `string`；第三种方式——`importlib.importmodule('string')` 内部调用 `import('string')` 并返回最终组件。三种方式殊途同归，但“返回值”所在的位置不同：

·设计思想：`import` 是静态的、可读的、可被工具分析的（依赖检查、打包工具都依赖它）；动态导入把这一环节推迟到运行时，付出可读性，换取灵活性。能用静态 `import` 就别用动态导入——这是两条路的取舍。

25.13 示例：传递式模块重载 (Example: Transitive Module Reloads)

英文原文

To tie together and apply some of the topics we've studied, this section develops a module tool that serves as a larger case study to close out this chapter and part. We explored module reloads in Chapter 23, as a way to pick up changes in code without stopping and restarting a program or REPL. When reloading a mod

ule, though, Python reloads only that particular module's file; it doesn't automatically reload modules that the file being reloaded happens to import.

For example, if you reload some module A, and A imports modules B and C, the reload applies only to A—not to B and C. The statements inside A that import B and C are rerun during the reload, but they just fetch the already loaded B and C module objects (assuming they've been imported before). In abstract code, here's the file A.py:

```
python # A.py import B # Not reloaded when A is! import C # Just imports of already loaded modules: no-ops
python >> from importlib import reload >> reload(A) # Doesn't reload B and C
```

By default, this means that you can't depend on reloads to pick up changes in all the modules in your program transitively. Instead, you must use multiple reload calls to update the subcomponents independently. This can require substantial work for large systems you're testing interactively. You can design your systems to reload their subcomponents automatically by adding reload calls in parent modules like A, but this complicates the code.

A recursive reloader A better approach is to write a general tool to do transitive reloads automatically, by scanning a module's dict attributes dictionary and checking the type of each attribute's value to find nested

d modules to reload. Such a utility function could call itself recursively to navigate arbitrarily shaped and deep import-dependency chains.

The module dict was introduced in Chapter 23 and employed by `mydir.py` earlier, recursion was explored in Chapter 19, and the type call was presented in Chapter 9; we just need to combine these tools for this new role.

To this end, the module `reloadall.py` listed in Example 25-7 defines a `reloadall` function that automatically reloads a module, every module that the module imports, and so on, all the way to the bottom of each import chain.

It uses a dictionary to keep track of already-reloaded modules, recursion to walk the import chains, and the standard library's `types` module, which simply predetermines type results for built-in types like modules.

Its visited dictionary avoids repeats when imports are recursive or redundant (module objects are immutable, and so can be dictionary keys); as we saw in Chapters 5 and 8, a set could work similarly (and will in rewrite ahead).

Example 25-7. `reloadall.py`

```
python """reloadall.py: transitively reload nested modules. Call reloadall with one or more imported modules as arguments. These modules, and all the modul
```

es they import, are reloaded.

```
"""
```

```
import types from importlib import reload def status(  
module): print('reloading', module.name) def tryreloa  
d(module): try: reload(module) # Imports might fail  
except: print('FAILED:', module) def transitive_reload(  
module, visited): if not module in visited: # Trap cycle  
s, duplicates status(module) # Reload this module try  
reload(module) # And visit children visited[module]  
= True  
  
for attr in module.__dict__.values(): # For all attrs in m  
od if type(attr) == types.ModuleType: # Recur if n  
ested module transitive_reload(attr, visited)  
  
def reloadall(args): visited = {} # Main entry point for  
arg in args: # For all passed in if type(arg) == types.  
ModuleType: transitive_reload(arg, visited)  
  
def tester(reloader, modname): # Self-test: cmd or pa  
ssed import importlib, sys # Imports for tests only if l  
en(sys.argv) > 1: # Command-line argument modna  
me = sys.argv[1] # Import by name string module = i  
mportlib.import_module(modname) # Import by name  
string reloader(module) # Test passed-in reloader  
  
if name == 'main': tester(reloadall, 'reloadall') # Test: r  
eload self or arg
```

Besides the module dict, this script makes use of other tools we've studied before: it includes a name test

to launch self-test code when run as a top-level script only, and its tester function uses `sys.argv` to inspect command-line arguments and `importlib` to import a module by name string passed in as a function or command-line argument. Review earlier coverage for more info if needed.

One curious bit: notice how this code's `tryreload` wraps the basic reload call in a try statement to catch exceptions. Reloads may fail for many reasons, and it's best to be defensive when using system interfaces. As you'll see in a moment, for example, an unreloadable monitoring module added to `sys` in Python 3.12 would otherwise crash the reloader. The try was previewed in Chapter 10 and will be covered in full in Part VII.

Testing recursive reloads To use this module normally, import its `reloadall` function and pass it an already loaded module object—just as you would for the built-in reload function. Like `reload`, its module argument is usually obtained by a top-level import; as we've seen, `sys.modules` fetches work too, but modules accessed only by `from` don't apply.

To test first, run the module standalone. Its tester function runs a passed-in reloader on a module imported by name string—which is taken from a command-line argument if present, else a passed-in name. In this mode, the module's self-test code calls `tester` to run r

reloadall on its own imported module by default if no command-line arguments are used (its own name is not defined in the file without an import):

```
python $ python3 reloadall.py reloading reloadall reloading types
```

With a command-line argument, the tester instead reloads the listed module by its name string—in the following, the Chapter 21 folder's benchmark module we coded in Example 21-8. To run this live, you need both the reloader module here and the module it reloads. One way to handle this is to add Chapter 21's code folder to your PYTHONPATH setting per Chapter 22.

Copying files in either direction is subpar, and simply running in the Chapter21 folder won't work because the reloader script's Chapter25 folder is "home." Note that we give a module name in this mode, not a filename; because the script imports the module using the search path just like import, the .py extension is omitted:

```
python $ pwd # In Chapter 25's code folder /Users/me/.../LP6E/Chapter25
```

```
$ export PYTHONPATH=../Chapter21 # Extend path:
your shell may vary $ python3 reloadall.py pybench #
Import+reload Chapter 21 module reloading pybench reloading sys reloading sys.monitoring FAILED: <module'sys.monitoring'> reloading os reloading abc re
```

loading stat reloading posixpath reloading genericpath
reloading time reloading timeit reloading gc reloading
itertools

More usefully, we can also deploy this module at the interactive prompt. This works like the built-in reload, but adds recursive reloads for the module or modules passed—here, for standard-library modules:

```
python $ python3 >> from reloadall import reloadall
# Reload stdlib modules in REPL mode >> import os,
tkinter >> reloadall(os) reloading os reloading abc re
loading sys reloading sys.monitoring FAILED: <modul
e'sys.monitoring'> reloading stat reloading posixpath
reloading genericpath >> reloadall(tkinter) reloading
tkinter reloading collections reloading collections.abc
... etc ... reloading sre reloading functools reloading c
opyreg
```

In either mode, the reloader also works on module packages—here, for the standard library's email package:

```
python $ python3 reloadall.py email.message # Import+reload a stdlib package reloading email.message r
eloading binascii reloading re ... etc ...
```

```
$ python3 >> from reloadall import reloadall # Same
, but in REPL mode >> import email.message >> reloadall(email.message) reloading email.message reloading binascii reloading re ... etc ...
```

The following runs the reloader on the `dir1.dir2.mod` path we coded in "Basic Package Structure." All items in the path are reloaded from a package root, and a `sys.path` mod in the REPL gives import access to another chapter's code folder—much like the `PYTHONPATH` setting used earlier (again, per Chapter 22):

```
python >> import sys >> sys.path.append('../Chapter
24') # Extend the search path manually >> import dir
1.dir2.mod # A package in Chapter 24's folder Running
dir1.init.py Running dir1.dir2.init.py Loading dir1.di
r2.mod >> reloadall(dir1) # Reloads all on path: mod
s in mods reloading dir1 Running dir1.init.py reloadin
g dir1.dir2 Running dir1.dir2.init.py reloading dir1.dir
2.mod Loading dir1.dir2.mod
```

Finally, here is a simple session that demos the effect of normal versus transitive reloads—changes made to the two nested files are not picked up by reloads unless our transitive utility is used (files are listed inline here for brevity):

```
python # File ra.py import rb X = 1 # File rb.py import
rc Y = 2 # File rc.py Z = 3 $ python3 >> import ra >
> ra.X, ra.rb.Y, ra.rb.rc.Z # Three-level import chain (1,
2, 3)
```

Now, without stopping Python, change all three files' assignment values, save the files, and reload back in the REPL:

```
python >> from importlib import reload >> reload(r
a) # Built-in reload is top-level only <module'ra' from
'../LP6E/Chapter25/ra.py'> >> ra.X, ra.rb.Y, ra.rb.rc.Z
(111, 2, 3) >> from reloadall import reloadall >> relo
adall(ra) # Normal usage mode reloading ra reloadin
g rb reloading rc >> ra.X, ra.rb.Y, ra.rb.rc.Z # Reloads
all nested modules too (111, 222, 333)
```

This is similar to the preceding package-path reload, but the imports here are explicit; module nesting in packages is implied. Study the reloader's code and results for more on its operation. The next section exercises its tools further.

Alternative codings For all the recursion fans in the audience (and we know who we are), Example 25-8 lists an alternative recursive coding for the original function in Example 25-7. This new version uses a set instead of a dictionary to detect repeats and cycles, is marginally more direct because it eliminates a top-level loop, and serves to illustrate recursive coding in general. Compare with the original to see how this differs.

This version also gets some of its work for free from the original; in fact, this module essentially extends the original to replace just the parts that vary. Notice how it calls the original version's tester, passing in the `reloadall` defined here—which ensures that this module's reloader is run when this script is launched in sta

ndalone mode.

Example 25-8. reloadall2.py

python """reloadall2.py: transitively reload nested modules. Alternative coding: recursive, refactored.

```
"""
```

```
import types from reloadall import status, tryreload, tester
def transitereload(objects, visited):
    for obj in objects:
        if type(obj) == types.ModuleType and obj not in visited:
            status(obj)
            tryreload(obj) # Reload this, recur to attrs
            visited.add(obj)
            transitereload(obj.dict.values(), visited)
```

```
def reloadall(args):
    transitereload(args, set())
```

```
if name == 'main':
    tester(reloadall, 'reloadall2') # Test: reload myself or arg
```

As we saw in Chapter 19, there is usually an explicit stack or queue equivalent to recursive functions, which may be preferable in some contexts. Example 25-9 lists one such a transitive reloader—it uses a stack instead of recursion, and a set to skip repeats and cycles

.

On each loop, all of a new module's attribute values are added to the end of the objects stack and filtered later, and this is repeated until the list of candidate objects becomes empty. A generator expression could filter out nonmodules in the extend call to avoid some pops, but this would be more complex.

Because it both pops and adds items at the end of its list, this version is stack-based, though the order of both pushes and dictionary values influences the order in which it reaches and reloads modules—it visits submodules in namespace dictionaries from right to left, unlike the left-to-right order of the recursive versions (trace through the code to see how). We could change this to match by reversing values, but reload order is unimportant.

Example 25-9. reloadall3.py

```
python """reloadall3.py: transitively reload nested modules. Alternative coding: nonrecursive, explicit stack.
```

```
"""
```

```
import types from reloadall import status, tryreload, tester
def transitivereload(objects, visited): while objects:
    next = objects.pop() # Delete next item at end if (
    type(next) == types.ModuleType # Is it a module object?
    and next not in visited): # Not already reloaded?
    status(next) # Reload this, push attrs
    tryreload(next)
    visited.add(next)
    objects.extend(next.dict.values())
```

```
def reloadall(args): transitivereload(list(args), set())
```

```
if name == 'main': tester(reloadall, 'reloadall3') # Test:
reload myself or arg
```

If the recursion and nonrecursion used in these examples is confusing, see the discussion of recursive functions in Chapter 19 for more background on the su

bject.

Testing reload variants To prove that these two alternative reloaders work the same as the original, let's test all three of our reloader variants. Thanks to their common testing function, we can run all three from a command line both with no arguments to test the module reloading itself, and with the name of a module to be reloaded listed on the command line (in `sys.argv`):

```
python $ python3 reloadall.py reloading reloadall reloading types
```

```
$ python3 reloadall2.py reloading reloadall2 reloading types
```

```
$ python3 reloadall3.py reloading reloadall3 reloading types
```

Though it's hard to see here, we really are testing the individual reloader alternatives—each of these tests shares a common tester function but passes it the `reloadall` from its own file. Here are the variants reloading the `tkinter` GUI module and all the modules its imports reach; again, the third's reload order varies:

```
python $ python3 reloadall.py tkinter reloading tkinter reloading collections reloading collections.abc ... etc ...
```

```
$ python3 reloadall2.py tkinter reloading tkinter reloading collections reloading collections.abc ... etc ...
```

```
$ python3 reloadall3.py tkinter reloading tkinter reloading re reloading copyreg ... etc ...
```

As usual, we can test interactively, too, by importing and calling either a module's main reload entry point with a module object, or the testing function with a reloader function and module name string:

```
python $ python3 >> import reloadall, reloadall2, reloadall3 >> import tkinter >> reloadall.reloadall(tkinter) # Normal use case reloading tkinter reloading collections reloading collections.abc ... etc ... >> reloadall.ltester(reloadall2.reloadall, 'tkinter') # Testing utility reloading tkinter reloading collections reloading collections.abc ... etc ... >> reloadall.tester(reloadall3.reloadall, 'reloadall3') # Mimic self-test code reloading reloadall3 reloading types
```

Finally, as noted, the third reloader's results will generally vary by order; reload order in all reloaders depends on namespace dictionary ordering (which, as we've learned, is deterministically ordered by key insertion time today), but the last also relies on the order in which items are added to its stack.

To ensure that all three are reloading the same modules irrespective of the order in which they do so, we can use sets or sorts to test for order-neutral equality of their printed messages—obtained here by running shell commands with the `os.popen` utility we used in Chapter 21:

```
python >> import os >> res1 = os.popen('python3 r
eloadall.py tkinter').readlines() >> res2 = os.popen('p
ython3 reloadall2.py tkinter').readlines() >> res3 = os
.popen('python3 reloadall3.py tkinter').readlines() >>
res1[:3] ['reloading tkinter\n','reloading collections\n'
,'reloading collections.abc\n'] >> res3[:3] ['reloading
tkinter\n','reloading re\n','reloading copyreg\n'] >> r
es1 == res2, res2 == res3 (True, False) >> set(res2) =
= set(res3) # Order-neutral equality True >> sorted(r
es2) == sorted(res3) # Ditto True
```

Run these scripts, study their code, and experiment on your own for more insight; these are the sort of importable tools you might want to add to your own source code library. Watch for a similar testing technique in the coverage of class tree listers in Chapter 31, where we'll apply it to passed class objects and extend it further.

Caveats: Keep in mind that all the transitive reloaders, like the reload built-in that they use, rely on the fact that module reloads update module objects in place, such that all references to those modules in any namespace will see the updated version automatically. Because from importers copy names out, they are not updated by reloads, transitive or not.

Perhaps worse, modules imported only by from won't be reloaded, because they do not exist in any importer's namespace scanned. Doing better may require e

either source code analysis or import customization.

Tool impacts like this are perhaps another reason to prefer import to from—which brings us to the end of this chapter and part, and the standard set of warnings for this part's topic.

中文翻译

为了把学过的一些主题串起来并加以应用，本节开发一个模块工具，作为本章和本部分的收官大案例。我们在第 23 章探索过模块重载 (module reloads) ——用它来拾取代码变更，而不必停止并重启程序或 REPL。不过，重载一个模块时，Python 只重载那一个模块的文件；它不会自动重载被重载文件所导入的模块。例如，如果你重载模块 A，而 A 导入了模块 B 和 C，那么重载只作用于 A——而不是 B 和 C。A 内部那些导入 B、C 的语句在重载时会重新执行，但它们只是去取已经加载过的 B、C 模块对象（假设它们之前被导入过）。用抽象代码表示，文件 A.py 长这样：

```
python # A.py import B # A 被重载时 B 不会重载! import C
# 只是导入已加载模块：空操作 python >>> from importlib import reload >>> reload(A) # 不会重载 B 和 C
```

默认情况下，这意味着你不能指望重载来传递性地拾取程序中所有模块的变更。相反，你必须多次调用 reload 来独立更新各个子组件。对于你在交互式环境里测试的大型系统，这可能要耗费大量工作。你也可以设计系统，让父模块（如 A）里添加 reload 调用来自动重载子组件——但这会把代码搞复杂。递归重载器 (A recursive reloader) 更好的做法是写一个通用工具来自动做传递式重载：扫描模块的 dict 属性字典，检查每个属性值的 type，找出嵌套的要重载的

模块。这样的工具函数可以递归地调用自己，遍历任意形状、任意深度的导入依赖链。模块 dict 在第 23 章引入、前面 mydir.py 刚用过；递归在第 19 章探讨过；type 调用在第 9 章讲过——我们只需要把这些工具组合起来，服务于这个新角色。为此，示例 25-7 列出的模块 reloadall.py 定义了 reloadall 函数：自动重载一个模块、该模块导入的每个模块、如此层层推进，一直到每条导入链的底端。它用字典记录已重载的模块，用递归走遍导入链，还用标准库的 types 模块——它只是预先定义了模块等内置类型的 type 结果。它的 visited 字典在导入递归或冗余时避免重复（模块对象是不可变的，因此可以作为字典键）；如我们在第 5 章和第 8 章所见，set 也能起到类似作用（后面改写时就会用到）。示例 25-7. reloadall.py

```
python """ reloadall.py
: 传递式地重载嵌套模块。调用 reloadall，并传入一个或多个已导入的模块作为参数。这些模块以及它们导入的所有模块都会被重载。"""
import types from importlib import
reload def status(module): print('reloading', module.
name) def tryreload(module): try: reload(module) #
导入可能失败except: print('FAILED:', module) def trans
itivereload(module, visited): if not module in visited:
# 拦截环、重复status(module) # 重载本模块tryreload(
module) # 然后访问其子模块visited[module] = True for
attr in module.__dict__.values(): # 遍历模块的所有属性if
type(attr) == types.ModuleType: # 若是嵌套模块则
递归transitivereload(attr, visited) def reloadall(args
): visited = {} # 主入口for arg in args: # 遍历传入的所有
参数if type(arg) == types.ModuleType: transitiverelo
ad(arg, visited) def tester(reloader, modname): # 自测
```

试：命令行或传入 `import importlib, sys` # 仅为测试而导入 `if len(sys.argv) > 1:` # 命令行参数? `modname = sys.argv[1]` # 按名字字符串导入 `module = importlib.import_module(modname)` # 按名字字符串导入 `reloader(module)` # 测试传入的重载器 `if name == 'main': tester(reloadall, 'reloadall')` # 测试：重载自己或参数除了模块 `dict`，这个脚本还用到了我们以前学过的其他工具：它包含 `name` 测试，仅当作为顶层脚本运行时才启动自测代码；它的 `tester` 函数用 `sys.argv` 检查命令行参数，用 `importlib` 按名字字符串导入模块（名字由函数参数或命令行参数传入）。需要的话可以回顾前面章节。一个有趣的细节：注意这段代码的 `tryreload` 用 `try` 语句包住了基本的 `reload` 调用以捕获异常。重载可能因许多原因失败，使用系统接口时最好采取防御姿态。比如，你马上就会看到：Python 3.12 中加进 `sys` 的、不可重载的 `monitoring` 模块，如果没有这个 `try`，就会把重载器搞崩溃。`try` 在第 10 章预习过，第七部分会完整讲解。测试递归重载（Testing recursive reloads）正常使用这个模块：导入它的 `reloadall` 函数，传入一个已加载的模块对象——就像你对内置 `reload` 函数做的那样。和 `reload` 一样，它的模块参数通常通过顶层的 `import` 获得；如我们所见，`sys.modules` 获取也可以，但只用 `from` 访问的模块不适用。先测试：单独运行这个模块。它的 `tester` 函数在一个按名字字符串导入的模块上运行传入的重载器——模块名来自命令行参数（若有），否则用传入的名字。在这种模式下，若没有命令行参数，模块的自测代码默认调用 `tester`，对它自己导入的模块运行 `reloadall`（不导入的话，自己的名字在文件里没有定义）：`python $ python3 reloadall.py reloading reloadall reloading types`

带命令行参数时，tester 改为按名字字符串重载列出的模块——下面是第 21 章代码文件夹里、我们在示例 21-8 编写的 benchmark 模块。要现场运行，你既需要这里的重载器模块，也需要它要重载的模块。一种做法是按第 22 章把第 21 章的代码文件夹加进你的 PYTHONPATH 设置。把文件往任何方向复制都是下策，而只在 Chapter21 文件夹里运行也不行，因为重载器脚本的 Chapter25 文件夹才是“老家”。注意：这种模式下我们给出的是模块名，不是文件名；因为脚本和 import 一样用搜索路径导入模块，所以 .py 扩展名要省略：python \$ pwd # 位于第 25 章的代码文件夹 /Users/me/.../LP6E/Chapter25 \$ export PYTHONPATH =../Chapter21 # 扩展路径：你的 shell 可能不同\$ python3 reloadall.py pybench # 导入并重载第 21 章的模块reloading pybench reloading sys reloading sys.monitoring FAILED: <module'sys.monitoring'> reloading os reloading abc reloading stat reloading posixpath reloading genericpath reloading time reloading timeit reloading gc reloading itertools 更有用的是，我们还可以把这个模块部署到交互式提示符（REPL）。它和内置 reload 一样工作，只是对传入的一个或多个模块增加了递归重载——这里重载的是标准库模块：python \$ python3 >>> from reloadall import reloadall # 在 REPL 模式重载标准库模块>>> import os, tkinter >>> reloadall(os) reloading os reloading abc reloading sys reloading sys.monitoring FAILED: <module'sys.monitoring'> reloading stat reloading posixpath reloading genericpath >>> reloadall(tkinter) reloading tkinter reloading collections reloading collections.abc...等等...reloading sre reload

adding functools reloading copyreg 无论哪种模式，重载器同样适用于模块包——下面重载标准库的 email 包：python \$ python3 reloadall.py email.message # 导入并重载标准库包reloading email.message reloading binascii reloading re...等等...\$ python3 >>> from reloadall import reloadall # 相同操作，REPL 模式>>> import email.message >>> reloadall(email.message) reloading email.message reloading binascii reloading re...等等... 下面在 "Basic Package Structure" (基本包结构) 一节编写的 dir1.dir2.mod 路径上运行重载器。路径中的所有条目都从包根开始重载；在 REPL 里修改 sys.path 让导入能访问另一章的代码文件夹——和前面用 PYTHONPATH 设置差不多（同样按第 22 章）：python >>> import sys >>> sys.path.append('../Chapter24') # 手动扩展搜索路径>>> import dir1.dir2.mod # 第 24 章文件夹里的一个包Running dir1.init.py Running dir1.dir2.init.py Loading dir1.dir2.mod >>> reloadall(dir1) # 重载路径上的所有：模块中的模块reloading dir1 Running dir1.init.py reloading dir1.dir2 Running dir1.dir2.init.py reloading dir1.dir2.mod Loading dir1.dir2.mod 最后，一个简单会话演示普通重载与传递式重载的差别——对两个嵌套文件做的修改，除非使用我们的传递式工具，否则重载不会拾取（文件为省篇幅在此内联列出）：python # 文件 ra.py import rb X = 1 # 文件 rb.py import rc Y = 2 # 文件 rc.py Z = 3 \$ python3 >>> import ra >>> ra.X, ra.rb.Y, ra.rb.rc.Z # 三层导入链(1, 2, 3) 现在，不停止 Python，把三个文件的赋值都改掉、保存文件，然后回到 REPL 重载：python >>> from importlib import reload >>> reload(

ra) # 内置 reload 只重载顶层<module'ra' from'./.../LP6E/Chapter25/ra.py'> >>> ra.X, ra.rb.Y, ra.rb.rc.Z (111, 2, 3) >>> from reloadall import reloadall >>> reload all(ra) # 正常使用模式reloading ra reloading rb reloading rc >>> ra.X, ra.rb.Y, ra.rb.rc.Z # 嵌套模块也全部重载 (111, 222, 333) 这和前面包路径的重载类似，但这里的导入是显式的；包里的模块嵌套是隐式的。仔细研究重载器的代码和结果，可以了解更多运行细节。下一节将进一步演练它的工具。替代写法 (Alternative codings) 献给在场所有递归爱好者（我们懂你们是谁）：示例 25-8 列出了示例 25-7 中原函数的替代递归写法。这个新版本用 set 代替字典来检测重复和环，因为去掉了顶层循环而略微更直接，同时也展示了通用的递归编码方式。与原版比较，看看差异在哪。这个版本还从原版免费得到了一部分工作；实际上，这个模块基本是扩展原版，只替换了变化的部分。注意它是如何调用原版的 tester，并传入这里定义的 reloadall 的——这保证了本脚本以独立模式启动时，运行的是本模块的重载器。示例 25-8. reloadall2.py python """ reloadall2.py: 传递式地重载嵌套模块。替代写法：递归、重构。""" import types from reloadall import status, tryreload, tester def transitereload(objects, visited): for obj in objects: if type(obj) == types.ModuleType and obj not in visited: status(obj) tryreload(obj) # 重载它，递归到其属性 visited.add(obj) transitereload(obj.dict.values(), visited) def reloadall(args): transitereload(args, set()) if name == 'main': tester(reloadall, 'reloadall2') # 测试：重载自己或参数如第 19 章所见，递归函数通常有显式的栈 (stack) 或队列 (queue) 等价物，在某些场景下可能

更可取。示例 25-9 列出了这样一个传递式重载器——它用栈代替递归，用 set 跳过重复和环。每次循环，一个新模块的全部属性值都被加到 objects 栈的末尾，之后再过滤，如此重复直到候选对象列表为空。生成器表达式可以在 extend 调用时就过滤掉非模块，省去一些 pop，但那样会更复杂。因为它既从列表末尾弹出又向末尾添加，这个版本是基于栈的；不过 push 的顺序和字典值的顺序都会影响它到达并重载模块的顺序——它按命名空间字典从右到左访问子模块，与递归版本从左到右的顺序相反（跟一遍代码就明白了）。我们可以通过反转 values 让它与递归版一致，但重载顺序并不重要。示例 25-9. reloadall3.py

```
python """ reloadall3.py: 传递式地重载嵌套模块。替代写法：非递归、显式栈。""" import types from reloadall import status, tryreload, tester def transitereload(objects, visited): while objects: next = objects.pop() # 删除末尾的下一项 if (type(next) == types.ModuleType # 是模块对象吗？ and next not in visited): # 还没重载过？ status(next) # 重载它，压入其属性 tryreload(next) visited.add(next) objects.extend(next.dict.values()) def reloadall(args): transitereload(list(args), set()) if name == 'main': tester(reloadall, 'reloadall3') # 测试：重载自己或参数如果这些示例里的递归与非递归写法让你困惑，参见第 19 章关于递归函数的讨论，那里有更完整的背景。测试重载变体 (Testing reload variants) 为了证明两个替代重载器和原版工作得一样，我们把三个重载器变体都测一遍。多亏了共用的测试函数，我们可以在命令行里分别运行三个脚本：既可以用无参数模式测试模块重载自身，也可以在命令行里 (sys.argv) 写上要重载的模块名：python $ python3 reloadall.
```

py reloading reloadall reloading types \$ python3 reloadall2.py reloading reloadall2 reloading types \$ python3 reloadall3.py reloading reloadall3 reloading types 虽然这里很难看出来，但我们确实在测试各个重载器变体——每个测试共用同一个 tester 函数，但传给它的是自己文件里的 reloadall。下面是各变体重载 tkinter GUI 模块及其导入链可达的全部模块；再次注意第三个的重载顺序有变化：python \$ python3 reloadall.py tkinter reloading tkinter reloading collections reloading collections.abc...等等...\$ python3 reloadall2.py tkinter reloading tkinter reloading collections reloading collections.abc...等等...\$ python3 reloadall3.py tkinter reloading tkinter reloading re reloading copyreg...等等...和往常一样，我们也可以交互式测试：导入后，要么用模块对象调用某个模块的主重载入口，要么用"重载器函数 + 模块名字符串"调用测试函数：python \$ python3 >>> import reloadall, reloadall2, reloadall3 >>> import tkinter >>> reloadall.reloadall(tkinter) # 正常用例reloading tkinter reloading collections reloading collections.abc...等等...>>> reloadall.tester(reloadall2.reloadall,'tkinter') # 测试工具reloading tkinter reloading collections reloading collections.abc...等等...>>> reloadall.tester(reloadall3.reloadall,'reloadall3') # 模拟自测代码reloading reloadall3 reloading types 最后，如前所述，第三个重载器的结果一般会按顺序变化；所有重载器的重载顺序都取决于命名空间字典的排序（我们学过，如今它按键插入时间确定性排序），但最后这个还取决于它往栈里添加条目的顺序。为了确保三个重载器不管以什么顺序都重载了相同的模块，我们可以

用 set 或排序来测试其打印消息的顺序无关相等性——这里用第 21 章用过的 os.popen 工具运行 shell 命令获得结果：

```
python >>> import os >>> res1 = os.popen('python3 reloadall.py tkinter').readlines() >>> res2 = os.popen('python3 reloadall2.py tkinter').readlines() >>> res3 = os.popen('python3 reloadall3.py tkinter').readlines() >>> res1[:3] ['reloading tkinter\n', 'reloading collections\n', 'reloading collections.abc\n'] >>> res3[:3] ['reloading tkinter\n', 'reloading re\n', 'reloading copyreg\n'] >>> res1 == res2, res2 == res3 (True, False) >>> set(res2) == set(res3) # 顺序无关的相等性 True >>> sorted(res2) == sorted(res3) # 同理 True
```

运行这些脚本、研读它们的代码，并自己动手实验，能获得更多洞见；这类可导入的工具正是你可能想收进自己源码库的东西。请留意第 31 章类树列出器（class tree listers）中类似的测试技巧——那里我们会把它应用到传入的类对象上并进一步扩展。注意事项（Caveats）：记住，所有传递式重载器——和它们使用的内置 reload 一样——都依赖这样一个事实：模块重载就地更新模块对象，于是任何命名空间中对这些模块的所有引用都会自动看到更新后的版本。因为 from 导入者把名字复制了出来，它们不会被子重载更新，无论传递与否。也许更糟：只通过 from 导入的模块根本不会被重载，因为它们不存在于任何被扫描的导入者命名空间中。要做得更好，可能需要源码分析或导入定制。这类工具影响或许是偏好 import 而非 from 的又一个理由——这也把我们带到了本章与本部分的末尾，以及本部分主题的标准警告清单。

深度理解

·核心概念：传递式重载（transitive reload）——内置 `reload` 只重载一个模块文件，不追它导入的子模块；`reloadall` 用“扫描 dict → 类型甄别 → 递归遍历”把整条导入链一次性刷新。这是对“模块图（module graph）遍历”的实战演练，也是本章前半段所有工具（`dict`、`type`、`sys.argv`、`importlib`、`name`）的大阅兵。

·底层实现：`reload` 的本质是“重新执行模块文件顶层语句，就地更新模块对象的 `dict`”；`transitivereload` 借此机制，把模块对象作为字典键存入 `visited`（模块对象可哈希，所以可行），用 `type(obj) == types.ModuleType` 精确命中“模块属性”；`types.ModuleType` 是 CPython 中模块类型的公开名称（`types` 模块由 C 扩展实现，直接导出了 `PyModuleType`）。三种实现：递归版（调用栈即遍历栈）、`reloadall2`（集合 + 参数即候选集）、`reloadall3`（显式 LIFO 栈，`pop()` 从末尾取、`extend` 推入，所以访问顺序是字典序的逆序）。

·设计原因：交互式开发（REPL 反复改库测试）中最痛的就是“改一个库，连带几十个模块要手动 `reload`”。作者给出一站式工具，同时示范“如何用基本构件拼出高级工具”——这正是 Python 哲学：没有内建功能时，语言给足原料让你自己造。

·实际问题：`sys.monitoring`（Python 3.12 新增，存放性能监控回调的模块）不可重载，这正是 `tryreload` 里 `try` 的用武之地——否则重载器会当场崩溃。`os.popen` 跑

子进程收集输出、用 set/sorted 做顺序无关比较，是"验证等价工具"的经典套路。

·初学者误区：① 以为 reload 会自动级联——不会，必须自己追链；② 以为 from 导出的名字也能被重载刷新——不能，from 是复制名字，重载后仍指向旧对象；③ 把 visited 想成"防止重复加载"——它防的是环（如模块互导）；④ 忽略模块对象可哈希性——dict/set 键要求哈希对象，模块对象天然满足；⑤ 裸 except: 捕获一切——作者故意如此（防御系统接口），但生产代码应写明异常类型。

代码分析 (Example 25-7、25-8、25-9)

```
def transitive_reload(module, visited):
    if not module in visited:
        status(module)
        tryreload(module)
        visited[module] = True
        for attrobj in module.__dict__.values():
            if type(attrobj) == types.ModuleType:
                transitive_reload(attrobj, visited)
```

·解释器如何处理：module.dict.values() 返回模块命名空间所有值的视图；type(attrobj) == types.ModuleType 逐个做精确类型比较（不是 isinstance! 所以只命中真正的模块对象，不命中其子类）。命中后递归深入。visited 以模块对象为键——dict 键要求哈希，模块对象默认哈希（其内存地址），所以能胜任；环结构（AB）会因 visited 检查而终止。

·reloadall2 的差异：把"候选模块"本身作为参数递归（tr

ansitivereload(args, set())) , 每个被重载模块再把 dict.values() 作为下一轮候选——省去顶层 for 循环。

· reloadall3 的差异: while objects: next = objects.pop() 是 LIFO 栈操作; objects.extend(next.dict.values()) 把子模块压栈; 因为是"后进先出", 遍历方向与字典顺序相反。三个版本最终重载的模块集合相同, 只是顺序不同——所以用 set(res2) == set(res3) 验证等价。

· 设计思想: 同一个任务给三种实现(递归、参数化递归、显式栈), 演示"算法等价而表达不同"。第 19 章说过: 递归能写的, 显式栈都能写; 选择哪种取决于可读性与控制力。

25.14 模块陷阱 (Module Gotchas)

英文原文

In this section, we'll explore the usual collection of boundary cases that can make life interesting for Python beginners. Some are review here, and a few are so obscure that coming up with representative examples can be a challenge, but most illustrate something important about the language.

中文翻译

本节我们探索那些"让 Python 初学者的生活变得有趣"的常见边界案例。有些是对旧知识的复习, 有些则冷门到连找代表例都颇费周章, 但大多数都阐明了语言中某个重要的东西

深度理解

·核心概念：陷阱 (gotchas) = 语义"看似合理实则坑人"的地方。模块世界里最密集的坑围绕两个点：名字 (namespace) 与重载 (reload)，而它们几乎都源于 from 的"复制而非链接"语义。

·底层实现：所有下面的陷阱都来自两条铁律：(1) 模块按文件从上到下顺序执行；(2) import 绑定模块对象、from 把名字复制进当前命名空间。理解了这两条，多数"意外"都在意料之中。

·生活例子：像"部门共享打印机"——from 像把文件复印一份带回自己工位（以后源文件怎么改都与你无关），import 像把打印机连到你电脑上（源头更新你立刻看到）。

25.14.1 模块名冲突：包与包相对导入 (Module Name Clashes: Package and Package-Relative Imports)

英文原文

If you have two modules of the same name, you may only be able to import one of them—by default, the one whose directory is leftmost in the sys.path module search path will always be chosen. This isn't an issue if the module you prefer is in your top-level script's directory; since that is always first in the module path

, its contents will be located first automatically. For cross-directory imports, however, the linear nature of the module search path means that same-named files can clash. To fix this, either avoid same-named files or use the package imports feature of Chapter 24. If you really need to get to two files of the same name, the latter is the solution: structure your source files in subdirectories, such that package-import directory names make the module references unique. As long as the enclosing package directory names are unique, you'll be able to access either or both of the same-named modules. This issue can also crop up if you accidentally use a name for a module of your own that happens to be the same as a standard-library module you need—your local module in the program's home directory (or another directory early in the module path) can hide and replace the library module. To fix that, either avoid using the same name as another module you need, or store your modules in a package directory and use the package-relative import model of Chapter 24. In this model, normal imports skip the package directory to access the library's version, but special dotted import statements can still select the local version of the module.

中文翻译

如果有两个同名模块，你可能只导得进来其中一个——默认情况下，搜索路径里最靠左的那个（`sys.path` 模块搜索

路径最左边目录里的那个) 总会被选中。如果你优先的那个模块就在顶层脚本的目录里, 这就不是问题——因为脚本目录总是模块路径的第一个, 它的内容会被最先找到。然而对于跨目录导入, 模块搜索路径的线性特性意味着同名文件可能冲突: 导入按顺序搜索路径, 先到先得, 后面的同名文件永远找不到。解决办法: 要么避免同名文件, 要么使用第 24 章的包导入 (package imports) 特性。如果你真需要访问两个同名文件, 后者才是答案: 把源文件组织进子目录, 让包导入的目录名使模块引用变得唯一。只要外层包目录名唯一, 你就能访问同名模块中的任何一个或两个。如果你无意中给自己模块起的名字恰好和你需要的某个标准库模块同名, 也会出现这个问题——你放在程序主目录 (或模块路径早期目录) 里的本地模块会"遮蔽"并取代库模块。解决办法: 要么避免使用你需要的其他模块的名字, 要么把模块放进包目录, 使用第 24 章的包相对导入模型 (package-relative import)。在该模型下, 普通导入会跳过包目录去访问库的版本, 而特殊的点号导入语句仍能选中本地版本的模块。

深度理解

·核心概念: `sys.path` 是有序列表, 同名模块"先到先得"。这是线性搜索路径的必然结果——跟"排队打饭"一样, 排第一个的把同名的全挡了。

·底层实现: `import` 时解释器按 `sys.path` 从左到右逐个目录尝试; 一旦某个目录里有匹配的 `.py/.pyd/.pyc` 文件 (或包 `init.py`), 立即停止搜索。所以同名文件只可能找到一个。

·设计原因：Python 没有"全盘注册表"式的模块系统，模块就是文件系统里的文件，定位靠路径查找而非 ID。这个设计简单、即插即用，代价就是"同名冲突"必须由用户自行规避。

·实际问题：最常见的受害者是"给你的模块起名叫 `math.py`、`json.py`"——它会在当前目录遮蔽标准库同名模块，导致库函数行为诡异。包结构（把代码放进 `mypkg/` 下）是官方推荐解法，因为包名成为模块名的前缀，天然消歧。

·初学者误区：以为文件夹里有个 `json.py` 没啥大不了，结果 `import json` 拿到的不是标准库。也误解"搜寻路径查找"会遍历所有可能——实际只要第一个命中就停止。

25.14.2 顶层代码的语句顺序很重要 (Statement Order Matters in Top-Level Code)

英文原文

As we've seen, when a module is first imported (or later reloaded), Python executes its statements one by one, from the top of the file to the bottom. This has a few subtle implications regarding forward references that are worth underscoring here:- Code at the top level of a module file (not nested in a function) runs as soon as Python reaches it during an import; because of that, it cannot reference names assigned lower in the file.- Code inside a function body doesn't run until

the function is called; because names in a function aren't resolved until the function actually runs, they can usually reference names anywhere in the file. In other words, forward references are usually only a concern in top-level module code that executes immediately; functions can reference names arbitrarily. Here's a file that illustrates forward reference dos and don'ts:

```
python func1() # Error: func1 not yet assigned
def func1():
    print(func2()) # OK: func2 looked up later
def func1(): # Error: func2 not yet assigned
def func2():
    return "Hello"
func1() # OK: func1 and func2 assigned
```

When this file is imported (or run as a standalone program), Python executes its statements from top to bottom. The first call to `func1` fails because the `func1` def hasn't run yet. The call to `func2` inside `func1` works as long as `func2`'s def has been reached by the time `func1` is called—and it hasn't when the second top-level `func1` call is run. The last call to `func1` at the bottom of the file works because `func1` and `func2` have both been assigned. Mixing defs with top-level code is not only difficult to read, but it's also dependent on statement ordering. As a rule of thumb, if you need to mix immediate code with defs, put your defs at the top of the file and your top-level code at the bottom. That way, your functions are guaranteed to be defined and assigned by the time Python runs the code that uses them.

中文翻译

如我们所见，模块首次被导入（或稍后被重载）时，Python 从上到下逐条执行其语句。这对前向引用（forward references）有几个值得在此强调的微妙影响：- 模块文件顶层的代码（不在函数内嵌套）在导入过程中 Python 一执行到它就运行；正因如此，它不能引用文件中更靠后才赋值的名字。- 函数体内的代码直到函数被调用才运行；因为函数里的名字直到函数真正运行才会被解析，所以它们通常可以引用文件中任何位置的名字。换句话说，前向引用通常只是“立即执行的顶层模块代码”才需要担心；函数可以任意引用名字。下面是一个文件，演示了前向引用的“该做”与“不该做”：

```
python func1() # 错误：func1 尚未赋值
def func1():
    print(func2()) # OK：func2 稍后才被查找
func1() # 错误：func2 尚未赋值
def func2():
    return "Hello"
func1() # OK：func1 和 func2 都已赋值
```

这个文件被导入（或作为独立程序运行）时，Python 从上到下执行其语句。第一次调用 func1 失败，因为 func1 的 def 还没执行。func1 内部调用 func2 只要在 func1 被调用前 func2 的 def 已经执行到就可以——而第二次顶层调用 func1 时 func2 尚未执行。文件末尾最后一次调用 func1 成功，因为此时 func1 和 func2 都已赋值。把 def 和顶层代码混在一起，不仅难以阅读，还依赖语句顺序。经验法则：如果你必须把立即执行的代码和 def 混在一起，把 def 放在文件顶部、顶层代码放到底部。这样，Python 运行使用这些函数的代码时，函数必已被定义并赋值。

深度理解

·核心概念：前向引用。"函数体内"与"顶层"两个区域对名字解析时机完全不同——顶层"即时解析"，函数体"用到才解析"。这是 Python 与 C/C++ 根本不同的地方：C 需要头文件/前置声明（前向声明）才能前向引用；Python 的函数体内天然允许。

·底层实现：Python 编译函数体时只把名字记录为 LOAD GLOBAL/LOADFAST 等延迟指令，真正查找发生在函数调用时。而顶层语句编译后立即执行，所以"下面还没定义"就是真的访问不到。执行是"流式编译+运行"，并非"全书符号表先建好"（C 语言的符号表先建会不同）。

·设计原因：模块是一次性顺序执行的过程模型——import 就"跑一遍文件"。之所以大多数语言允许前置引用，是因为它们分"编译期符号表"与"运行期"两步；Python 选择极简的"边编边跑"，代价就是顶层前向引用受限。

·实际问题：大型模块里把"大段顶层逻辑"放在 import 之后是坏味道；正确结构是"import → 定义 → main 入口"，这让文件既可被导入又可直接运行（回到 25.7 的 name 技巧）。

·初学者误区：以为是"变量作用域"问题。其实完全不是作用域——是"语句执行顺序"问题：同样名字在后面定义了，前面顶层代码也看不到。理解这一点才能不看文档就推断 def 与顶层代码混排的后果。

25.14.3 复制名字的 from: 拷贝而不链接 (from Copies Names but Doesn't Link)

英文原文

Although it's commonly used, the `from` statement is the source of a variety of potential gotchas in Python. As we've seen, the `from` statement is really an assignment to names in the importer's scope—a name-copy operation, not a name aliasing. The implications of this are the same as for all assignments in Python, but they're especially subtle for names that live in different files. As a refresher, suppose we define the simple module `nested.py` in Example 25-10. Example 25-10.

```
nested.py python X = 99 def printer(): print(X)
```

If we import its two names using `from` in another module (or the REPL, which stands in for one), we get copies of those names, not links to them. Changing a name in the importer resets only the binding of the local version of that name, not the name in `nested.py`:

```
python >>> from nested import X, printer # Copy names out
>>> X = 88 # Changes my X only! >>> printer() # nested.py's X is still 99 99
```

If we instead use `import` to get the whole module and assign to a qualified name, we change the name in `nested` (the module's loaded image, not its source code). Attribute qualification directs Python to a name in the module object, rather than a name in the importer:

```
python >>> import nest
```



```
ed # Get module as a whole >>> nested.X = 88 # Change nested.py's X >>> nested.printer() 88
```

Takeaway: changes to names obtained with `from` don't impact any other module. As covered in Chapter 23, changes to mutable objects shared by names copied with `from` can impact other modules, but name changes cannot.

中文翻译

尽管被广泛使用，但 `from` 语句在 Python 里是各种潜在陷阱的源头。如我们所见，`from` 语句其实是对“导入方作用域里名字”的赋值——是名字拷贝操作，而不是名字别名（aliasing）。其含义与 Python 里所有赋值相同，但尤其容易在“名字分属不同文件”时让人踩坑。作为复习，假定我们在示例 25-10 定义简单的模块 `nested.py`。示例 25-10.

```
nested.py python X = 99 def printer(): print(X)
```

如果我们在另一个模块（或 REPL）里用 `from` 导入它的两个名字，我们得到的是这些名字的拷贝，而不是链接。在导入方修改某个名字，只重置那个名字在本地的版本，而不影响原模块里的名字：

```
python >>> from nested import X, printer # 把名字复制出来 >>> X = 88 # 只改我的 X! >>> printer() # nested 的 X 仍是 99
```

99 如果改用 `import` 拿到整个模块，再对限定名赋值，我们改的是模块里的名字（模块已加载的映像，不是源码）；这个修改对任何使用该模块的人都生效：

```
python >>> import nested # 把模块整体拿到手 >>> nested.X = 88 # 修改 nested 的 X >>> nested.printer() 88
```

要点：名字拷贝只是快照，不是活链接。用 `from` 得到的名字，其改动不影响任何其他模块。如第 23 章所述

，用 `from` 复制出的名字所共享的可变对象的改动能影响其他模块，但名字的改动不能。

深度理解

·核心概念：“复制”vs“链接”。`from nested import X` 之后再 `X = 88`，只是给本地 `X` 重新绑定一个对象；原模块依旧有旧对象。`import nested; nested.X = 88` 则是在模块命名空间里改绑定——所有引用 `nested.X` 处都变了。

·底层实现：`from` 编译为 `IMPORTNAME` 后紧跟一个赋值（`STORE`），把模块属性值复制给新名字。之后本地 `X` 与 `nested.X` 只是“初值相同”的不相干两个名字。重新绑定本地名（`X = 88`）不会回头改动模块字典。

·设计原因：Python 里 `X = 88` 是重新绑定名字到新对象，从不就地改旧对象本身。这让“值”与“变量”彻底分离——也就让 `from` 的“快照”语义显得意外（但实则必然）。

·实际问题：这是“reload 对 `from` 无效”以及本章传递式工具对“`from-only` 模块”失效的根因。也解释了为何 `import x as y` 比 `from x import` 更“重载友好”。

·初学者误区：把 `X = 88` 想成“把 `nested.X` 改成 88”——不是；它只是在你的作用域里新建了一个 `X`。同理 `lst.append()` 是就地改写对象（共享可见），而 `lst = [...]` 是重新绑定（只影响本地名）——这条区别决定了“`from` 能否穿透”。

代码分析 (Example 25-10)

```
X = 99
```

```
def printer(): print(X)
```

·解释器如何处理：导入 `nested.py` 时，模块命名空间里绑定 `X → 99`、`printer → 函数对象`。`from nested import X, printer` 时，解释器去 `sys.modules['nested']` 的 `dict` 里取这两个值，把引用复制到导入方命名空间。此后两套名字指向同一堆对象；但一旦导入方出现 `X = 88`，只是让本地 `X` 换绑到 `88`——`nested.X` 纹丝不动，`printer()` 打印的是嵌套函数全局命名空间里的 `X`，仍然是 `99`。

·设计思想："名字绑定对象、赋值即重绑"的底子意味着 `from` 永远做不到"活链接"。想"活的"就别复制，用 `import module + 属性访问`——这也是 `PEP 8` 风格指南倾向 `import` 的原因。

25.14.4 `from` 可能遮蔽变量的含义 (`from Can Obscure the Meaning of Variables`)

英文原文

This was mentioned earlier but its demo was saved for here. Because you don't list the variables you want when using the `from` statement form, it can accidentally overwrite names you're already using in your scope. Worse, it can make it difficult to determine where a variable comes from. This is especially true if the `from` form is used on more than one imported file. For example, if you use the `from` form on the three modules in the following, you'll have no way of knowing w

that a raw function call really means, short of searching all three external module files—all of which may be in other directories: `python >>> from module1 import # May overwrite names silently >>> from module2 import # No way to tell what we get >>> from module3 import # No way to see name origins >>> func()` # Huh? The solution is simply not to do this: list the attributes you want in most from statements, and use at most one from per file. That way, any undefined names must by deduction be in the module named in the single from . You can avoid the issue altogether if you always use import instead of from, but that advice is too harsh; like much else in programming, from is a convenient tool if used wisely. Even this example isn't an absolute evil—it's OK for a program to use this technique to collect names in a single module for convenience, as long as it's well known.

中文翻译

这在前面提过，但它的演示留到了这里。因为用 from 语句形式时你不列出想要的变量，它可能意外覆盖你已经在自己作用域里用着的名字。更糟的是，它让你难以判断一个变量从哪来。如果在多个导入文件上都用 from，问题尤其突出。例如，对下面的三个模块依次使用 from 后，你基本无从知晓一个光秃秃的函数调用到底指代什么，除非去搜遍三个外部模块文件——而它们可能都在别的目录里： `python >>> from module1 import # 可能静默地覆盖名字 >>> from module2 import # 说不出我们得到了什么 >>> from`

module3 import # 看不出名字来源>>> func() # 啊? 解决办法很简单, 就是别这么干: 大多数 from 语句里都列出你想要的属性, 每个文件至多用一次 from。这样一来, 任何未定义的名字, 按排除法必然是来自那唯一一次 from 所指的那个模块。如果永远用 import 而不用 from, 你可以完全避开这个问题——但这建议太死板了; 和编程里许多事情一样, from 用聪明了是个很方便的工具。连这个例子也并非绝对之恶——只要大家都知道, 让程序用这种技巧把名字收集进单个模块图个方便, 也未尝不可。

深度理解

- 核心概念: from 的最大罪名不是"导入了太多" (可配 all/X 控制), 而是砍断名字的可追溯性——你无法从代码本身判断名字来源。
- 底层实现: from 在字节码上是 IMPORTSTAR 指令, 运行时把模块 dict 中 (受 all/下划线规则控制) 的所有名字复制进当前作用域, 其中覆盖同名旧绑定。它不会报重复定义的错——就是闷声覆盖。
- 设计原因: from 设计初衷是"交互式/脚本式快速取用", 不是生产库的标配。语言的自由哲学允许它存在, 但"显式优于隐式"又想提醒你别依赖它。
- 实际问题: 大型代码库 lint (flake8 F403/F405) 与类型检查器 (mypy) 实现都不好处理 from; 它也让 IDE 的"跳到定义"失效。团队规范里通常一条"禁止除 init.py 再导出外的 from"。
- 初学者误区: 以为 from 只是"方便省事"。它最致命的是

把错误变难查——同名覆盖后，行为诡异却无任何提示；多文件 `from` 之后唯一的重载是把作用域清理成"只留 `all` 白名单"的做法。

代码分析

```
>>> from module1 import *      #
>>> from module2 import *      #
>>> from module3 import *      #
>>> func()                     # func
```

·解释器如何处理：三条 `IMPORTSTAR` 依次执行，把三个模块导出的名字（非 `X/受 all` 限制）逐个绑定进当前命名空间，后者覆盖前者。到 `func()` 时解释器在本作用域找到 `func` 名字，直接调用——至于它是 `module1/module2/module3` 谁的 `func`，解释器和程序员一样无从得知（取决于导出顺序与覆盖次序）。

·设计思想：命名空间是"薄薄一层字典"。`from` 是唯一一种"无清单合并"，等于把"你是谁"的答案交给运气——违背可读性优先的语言气质。

25.14.5 reload 可能不影响 from 导入 (reload May Not Impact from Imports)

英文原文

Here's another from-related gotcha: as discussed previously, because `from` copies (assigns) names when run, there's no link back to the modules where the names came from. Names imported with `from` simply b

ecome references to objects, which happen to have been referenced by the same names in the importee when the from ran. Because of this behavior, reloading the module of origin has no effect on clients that import its names using from. That is, the client's names will still reference the original objects fetched with the from, even if the names in the original module are later reset. Here's the story in abstract code:

```
python from module import X # X may not reflect any module reloads ...
from importlib import reload
reload(module) # Changes module, but not my names X
# Still references old object!
```

To make reloads more effective, use import and name qualification instead of from. Because qualifications always go back to the module, they will find the new bindings of module names after reloading has updated the module's content in place:

```
python import module # Get module, not names ...
from importlib import reload
reload(module) # Changes module in place
module.X # Get current X: reflects module reloads
```

This is why our transitive reloader earlier in this chapter doesn't apply to names fetched with from, only import; again, if you're going to use reloads, you're probably better off with import.

中文翻译

这是另一个与 `from` 相关的陷阱：正如前面所述，因为 `from` 在加载时是拷贝（赋值）名字，所以它与“名字来源的模块”之间没有链接。用 `from` 导入的名字只是变成对对象的引用——这些对象恰好是在 `from` 执行时被被导入方（`importee`）里同名的名字所引用的东西。正因为这个行为，重载“来源模块”对用 `from` 导入其名字的客户端没有任何效果。也就是说：客户端的名字仍引用最初 `from` 取到的原始对象，即使原始模块里的名字后来被重置。抽象代码的故事如下：
`python from module import X # X 不反映任何模块重载... from importlib import reload reload(module) # 改了模块，但改不了我的名字X # 仍引用旧对象！`
让重载更有效的方法是：用 `import + 名字限定 (name qualification)`，而不是 `from`。因为限定每次都会回到模块，重载把模块内容就地更新后，它总能找到模块名字的最新绑定：
`python import module # 拿到模块而不是名字... from importlib import reload reload(module) # 就地更新模块 module.X # 取当前 X：反映重载结果这正是本章前面那个传递式重载器不适用于 from 取来的名字、只适用于 import 的原因；再说一次，如果你要用重载，最好还是用 import。`

深度理解

·核心概念：`reload` 就地（`in place`）更新模块对象，但 `from` 只拷贝引用；所以重载后，“模块.名字”能看到新值，“导出的名字”仍是旧值。这是“名字拷贝非链接”的直接推论。

- 底层实现：reload(module) 的机制是：重新执行模块源码，把结果写回现有模块对象的 dict（不换对象，id(module) 不变）。module.X 每次读取都查字典——看到新值；from module import X 在某时刻复制了对象引用——之后字典内容怎么变都改不了已存的引用。
- 设计原因：这保证"已有代码的稳定性"——重载不会撕裂正在运行的系统。Python 明确文档说明 reload 不能伸缩到对已有 from 引用生效，说明"它只是更新模块的命名空间"这一精确语义。
- 实际问题：调试/开发热更新（热重载 web 服务器）时——你改模块文件+reload，但要测的 from mymod import util 仍是旧版，烦恼正是这机制。
- 初学者误解：以为 reload 会和 from 下的所有名字"联动"。它只与 module.name 读取路径联动——所以热重载框架（如 hupper、watchfiles）会强制你"重启进程或再次 import"。

代码分析

```
from module import X          # X copies at import time
...
from importlib import reload
reload(module)                # module.__dict__
X                             # X
import module
module.X                      # →
```

- 工作机制：from module import X 等价于 X = module.dict['X']，是一次引用复制。reload 改写模块字典，但已复制的引用不动。对比两种读取，一旧一新在同一时刻并

存——这是"名字复制 vs 字典查找"两种访问方式的时差

。

·工程意义：凡用 reload 的地方，代码里一律用 import module; module.X，可少踩无数个坑。

25.14.6 reload、from 与交互式测试 (reload, from, and Interactive Testing)

英文原文

In fact, the prior gotcha is even more nuanced than it appears. Chapter 3 warned that it's usually better not to launch programs with imports and reloads because of the complexities involved. Things get even worse when from is brought into the mix. Python beginners most often stumble onto its issues in scenarios like this—imagine that after opening a module file in a text edit window, you launch an interactive session to load and test your module with from:

```
python from module import function function(1, 2, 3)
```

Finding a bug, you jump back to the edit window, make a change, and try to reload the module this way:

```
python from importlib import reload reload(module)
```

This doesn't work, because the from statement assigned only the name function, not module. To refer to the module in a reload, you have to first bind its name with an import statement at least once:

```
python from importlib import reload import module
reload(module) function(1, 2, 3)
```

But this doesn't quite work either—reload does update the module object, but as discussed in the previous section, the name function still refers to the old object; it's not automatically rebound. To really get the new function, you must refer to it as module.function after the reload, or rerun the from:

```
python from importlib import reload import module
reload(module)
```

```
from module import function # Or give up and use module.function()!
function(1, 2, 3)
```

Now, the new version of the function will finally run, but it seems an awful lot of work to get there.

中文翻译

实际上，前面的陷阱比它看起来更糟。第 3 章警告过，因为其中牵涉的复杂性，最好不要用 imports 和 reloads 来启动程序。而一旦把 from 掺进来，事情会变得更麻烦。初学者最容易在下面的场景撞上它——想象一下：你在文本编辑窗口打开一个模块文件，然后启动一个交互式会话，用 from 加载并测试模块：python from module import function function(1, 2, 3) 你发现 bug，跳回编辑窗口改了代码，然后想这样 reload 模块：python from importlib import reload reload(module) 这不行——因为 from 语句只赋值了 function 这个名字，没有赋值 module。要在 reload 里引用模块，你至少得先用一次 import 把模块名

绑定: `python from importlib import reload import module reload(module) function(1, 2, 3)` 但这也还是不行——`reload` 确实更新了模块对象, 但正如上一节所说, `function` 这个旧名字仍然指向旧对象(引用未重绑)。要拿到新函数, 你必须在 `reload` 之后写成 `module.function`, 或者重新执行一次 `from`: `python from importlib import reload import module reload(module) from module import function # 或者干脆放弃, 用 module.function()` ! `function(1, 2, 3)` 现在, 函数的新一版总算跑起来了, 可走到这一步, 实在有点大费周章。

深度理解

- 核心概念: 交互式 (REPL) 改代码 → `reload` 的完整 workflow, 如何被"名字复制"语义搅乱, 以及必须依赖 `import` + 限定名才能达到预期。
- 底层实现: REPL 顶层代码也在 `main` 模块命名空间执行, 行为与脚本一致。`from module import function` 三连 (`import+reload+显式 from`) 是唯一语义化路径。
- 设计原因: 没有"自动探测文件变化并生效"的年代, `reload` 是唯一实时的开发口; Python 把它设计得"模块对象就地更新", 为此文档明示局限: 不保证既有名字/引用同步。
- 实际业务: 现代服务器热重载也要面对同样 ASCII (进程内 `reload` 需要手工清理缓存、重建 `import`) ; `idealize` 就是为什么社区普遍推荐"保存→重启进程"而非 `reload`。
- 进阶: `<module>.X` 每次求值都重新查模块字典——这

是 reload 能生效的唯一破坏面。而 id() 三个关键词足以看清一切。

代码分析

```
from module import function      #  
reload(module)                   # function  
    module.function(1,2,3)      #
```

·解释器如何处理：reload 执行模块源码并 STORE 回模块字典；module.function 是"属性查找"，每次从字典重取——所以拿到新版；而旧 function 名字是执行期复制的引用，不会变。

·总结一句话：reload 更新的是模块的字典，不是你的绑定。要吃到更新，走上"module"这条路。

25.14.7 递归 from 导入可能不工作 (Recursive from Imports May Not Work)

英文原文

The most bizarre (and, thankfully, obscure) gotcha has been saved for last. Because imports execute a file's statements from top to bottom, you need to be careful when using modules that import each other. This is often called recursive imports, but the recursion doesn't really occur (in fact, circular may be a better term here)—such imports won't loop infinitely. Still, because the statements in a module may not all have been run when it imports another module, some of its n

ames may not yet exist. If you use `import` to fetch the module as a whole, this probably doesn't matter; the module's names won't be accessed until you later use qualification to fetch their values, and by that time the module is likely complete. But if you use `from` to fetch specific names, you must bear in mind that you will only have access to names in that module that have already been assigned when the recursive import occurs. For example, consider the modules in Examples 25-11 and 25-12, `recur1.py` and `recur2.py`:

Example 25-11. `recur1.py`

```
python X = 1
import recur2 # Run recur2 now if it doesn't exist
Y = 2
```

Example 25-12. `recur2.py`

```
python from recur1 import X # OK: X already assigned
from recur1 import Y # Error: Y not yet assigned
```

Module `recur1` assigns a name `X` and then imports `recur2` before assigning the name `Y`. At this point, `recur2` can fetch `recur1` as a whole with an `import`—it already exists in Python's internal modules table, which makes it importable, and also prevents the imports from looping. But if `recur2` uses `from`, it will be able to see only the name `X`; the name `Y`, which is assigned below the `import` in `recur1`, doesn't yet exist, so you get an error:

```
python $ python3 >>> import recur1
ImportError: cannot import name 'Y' from partially initialized module 'recur1' (most likely due to a circular import) (/.../LP6E/Chapter25/recur1.py)
```

Python avoids rerunning `recur1`'s statements when they are imported recursively from `recur2` (otherwise the imports would

send the script into an infinite loop that might require a Ctrl+C solution or worse), but recur1's namespace is incomplete when it's imported by recur2. The solution? Don't use from in recursive imports (no, really!). Python won't get stuck in a cycle if you do, but your programs will once again be dependent on the order of the statements in the modules. In fact, there are two ways out of this gotcha:- You can usually eliminate import cycles like this by careful design—maximizing cohesion and minimizing coupling per the start of this chapter are good first steps.- If you can't break the cycles completely, postpone module name accesses by using import and attribute qualification (instead of from and direct names), or by running your froms either inside functions (instead of at the top level of the module) or near the bottom of your file to defer their execution. There is additional perspective on this issue in the exercises at the end of this chapter—which we've officially reached.

中文翻译

最怪诞（也幸亏罕见）的陷阱被留到了最后。因为导入是从上到下执行文件语句，所以当模块之间互相导入时，你要格外小心。这常被叫做递归（recursive）导入，但真正的递归并没有发生（事实上用循环（circular）这个词更恰当）——这类导入并不会无限循环。只不过：一个模块导入另一个模块时，它自己的语句可能还没全部跑完，于是其中一些名字可能还不存在。如果你用 import 把模块整体取下来

，这通常没关系——模块里的名字要到后期你用限定名 (qualification) 去取时才访问，到那个时候模块基本已经执行完整了。但如果用 from 来取具体名字，你必须记住：你只能访问到循环导入发生时、在那个模块里已经被赋过值的名字。考虑示例 25-11 与 25-12 的演示：recur1.py 与 recur2.py：示例 25-11. recur1.py python X = 1 import recur2 # 若还不存在，此刻运行 recur2 Y = 2 示例 25-12. recur2.py python from recur1 import X # OK: X 已赋值 from recur1 import Y # 错误: Y 尚未赋值模块 recur1 先赋 X，然后（在赋 Y 之前）导入 recur2。此刻 recur2 可以用 import 把 recur1 整体拿到——因为 recur1 已经出现在 Python 内部模块表 (internal modules table) 里，这使它可被导入，也阻止了导入无限循环。但如果 recur2 用 from，它只能看到 X；名字 Y 是在 recur1 中 import 语句之后才赋值的，此刻还不存在，于是报错：python \$ python3 >>> import recur1 ImportError: cannot import name 'Y' from partially initialized module 'recur1' (most likely due to a circular import) (/.../LP6E/Chapter25/recur1.py) 当 recur2 里递归地导入 recur1 时，Python 会避免重跑 recur1 的语句（否则导入会把脚本送进无限循环，可能需要 Ctrl+C 甚至更糟才能脱身），可当 recur2 导入它时，recur1 的命名空间还是不完整的。解决办法？在递归导入中不要用 from（真的，别用！）。即使用了 Python 也不会卡死在循环里，但你的程序就会再次依赖模块中语句的顺序。实际上，有两条路摆脱这个陷阱：- 通常可以通过仔细的设计消除此类导入循环——本章开头所说的最大化内聚、最小化耦合，就是良好的第一步。- 如果无法彻底断开循环，就推迟模块名访问：改用 import + 属

性限定（而不是 `from + 直接名字`），或者把你的 `from` 放进函数体内（而不是模块顶层），或放在文件靠底部的位置，以延迟其执行。这个问题在本章末尾的练习里还有另一重视角的讨论——而我们已经正式抵达本章末尾了。

深度理解

·核心概念：循环导入（circular import）与它的真面目——不是“无限循环”，而是“命名空间不完整”：A 导 B、B 反导 A 时，A 只执行了一半，`from A import X` 可能取不到尚未创建的 X。

·底层机制：导入系统在 `sys.modules` 里先占位登记再执行代码（防止自递归）。当 `recur2` 反向导入 `recur1` 时，`sys.modules['recur1']` 已存在（半成品），Python 不会重新执行它，直接把半成品返回。于是 `recur1` 中尚未执行到的 Y 就不存在 → `ImportError: cannot import name 'Y' from partially initialized module ... (most likely due to a circular import)`。

·为什么不会死循环：`sys.modules` 的占位符是“防回流闸”——每个模块只执行一遍。真正的坑不是循环本身，而是半成品期从它那取名字。

·实际问题：大型项目模块互导很常见（如 ORM 模型、插件注册）。解法优先级：①重构消灭循环；②延迟绑定（函数内 `import` / 文件底部 `from` / 用 `import + 限定`）。许多框架（Django、SQLAlchemy）也大量用“注册表 + 延迟导入”避开此坑。

·初学者误区：以为“循环导入 = 一定报错”。其实 `import`

的姿态极其宽容——你从来不立刻取名字，等用的时候模块早完整了。真正会炸的是顶层 `from + 时间差`。也别随便在 `sys.modules` 里手动删项来“绕坑”，那往往是自找更多麻烦。

代码分析 (Example 25-11 与 25-12)

```
# recur1.py
X = 1
import recur2                # recur2
Y = 2                        # Y recur2 ""
```

```
# recur2.py
from recur1 import X          # OKrecur1 X=1
from recur1 import Y          # ERRORY recur1 → ImportError
```

·解释器如何处理：`import recur1` → 先在 `sys.modules` 登记半成品 (partially initialized) → 执行顶层：`X=1`；执行到 `import recur2` 时暂停 `recur1`，进入 `recur2` 执行其顶层：第一条 `from recur1 import X`——查 `sys.modules['recur1']` (已在，不重跑)，从字典取 `X` (已存在) → 成功；第二条 `from recur1 import Y`——字典里此刻没有 `Y` (`recur1` 尚未执行到 `Y=2`) → 抛 `ImportError`，错误信息精确点出 “partially initialized module”。

·若用 `import recur1` (而非 `from`) 呢？取回半成品模块对象本身没问题——名字访问推迟到你代码里写 `recur1.Y` 的时候 (那时模块早已完整)。这就是“延迟访问”能救场的原理。

25.15 本章小结 (Chapter Summary)

英文原文

This chapter surveyed module topics, some of which qualify as advanced. We studied data-hiding techniques, enabling new language features with the future module, the name usage-mode variable, transitive reloads, importing by name strings, and more. We also explored module design issues, wrote some substantial programs, and looked at common mistakes related to modules to help you avoid them in your code. The next chapter begins our exploration of Python's class—its object-oriented programming tool. Much of what we've covered in the last few chapters will apply there, too: classes live in modules and are namespaces as well, but they add an extra component to attribute lookup called inheritance search. As this is the last chapter in this part of the book, however, before we dive into classes, be sure to work through this part's set of lab exercises. And before that, here is this chapter's quiz to review the topics covered here.

中文翻译

本章对模块主题做了全景式盘点，其中一些内容称得上高级。我们学习了数据隐藏技巧、用 `future` 模块启用新语言特性、`name` 用途模式变量、传递式重载、按名字字符串导入，等等。我们还探讨了模块设计问题、编写了一些颇有分量

的程序，并查看了与模块相关的常见错误，帮你避免在自己的代码里踩坑。下一章将开启我们对 Python 类（class）——它的面向对象编程工具——的探索。前几章学到的大部分内容在那里同样适用：类活在模块里，本身也是命名空间，但它们在属性查找上额外增加了一个成分，叫做继承搜索（inheritance search）。不过，因为这是本部分书的最后一章，在潜入类之前，务必完成本部分的实验练习。而在这之前，先来一份本章测验，回顾一下这里涉及的主题。

深度理解

- 核心概念：本章是“模块知识全景图”——数据隐藏、双模式、动态导入、传递式重载、陷阱；总结一句话：模块 = 命名空间 + 文件 + 生命周期（导入/重载）。
- 承上启下：下一章（第 26 章）进入类（class）——类同样活在模块里、同样是命名空间，只是额外叠加了“继承搜索”。这是本书最经典的“预告片”：模块是“扁平命名空间”，类是“带继承链的命名空间”。
- 复习主线：25.3 约定式隐藏 → 25.6 future 时间旅行 → 25.7 name 双模式 → 25.12 动态导入 → 25.13 重载器（把一切串起来）→ 25.14 陷阱（把一切坑说透）。整章其实是“模块三部曲”的答辩状。
- 学习建议：按“能否一句话向别人解释”自测：all vs X、import vs importmodule、reload vs reloadall，这三对差异是本章含金量最高的考点。

25.16 自测：测验与答案 (Test Your Knowledge: Quiz / Answers)

英文原文

1. What is significant about variables at the top level of a module whose names begin with a single underscore? 2. What does it mean when a module's name variable is the string 'main'? 3. How might you step through all the attributes in a module with a loop? 4. If the user interactively types the name of a module to test, how can your code import it? 5. If the module future allows us to import from the future, can we also import from the past? Answers: 1. Variables at the top level of a module whose names begin with a single underscore are not copied out to the importing scope when the from statement form is used. They can still be accessed by an import or the normal from statement form, though. The all list is similar, but the logical converse; its contents are the only names that are copied out for a from . 2. If a module's name variable is the string 'main', it means that the file is being executed as a top-level script instead of being imported from another file in the program. That is, the file is being used as a program, not a library. This usage-mode variable supports dual-mode code and tests. 3. By using the module's built-in dict attribute. This is a normal dictionary that holds all of the module's attributes,

so code can iterate over its keys, values, or key/value pairs. When needed, attribute values can also be fetched for string names by indexing dict, or calling the getattr built-in function. 4. User input usually comes into a script as a string; to import the referenced module given its string name, you can build and run an import statement with exec, or pass the string name in a call to the import or importlib.importmodule functions. 5. No, we can't import from the past in Python. We can install (or stubbornly use) an older version of the language, but the latest Python is generally the best Python (with apologies to 2.X fans in the audience).

中文翻译

1. 模块顶层那些以单个下划线开头的名字有什么特别之处？ 2. 当模块的 name 变量是字符串'main' 时意味着什么？ 3. 你如何用一个循环遍历模块的所有属性？ 4. 如果用户交互式地输入了一个要测试的模块名，你的代码该如何导入它？ 5. 既然 future 模块让我们能"从未来导入"，我们能"从过去导入"吗？ 答案： 1. 模块顶层那些以单个下划线开头的名字，在使用 from 语句形式时不会被复制到导入方作用域。不过它们仍然可以用 import 或普通 from 语句形式访问。all 列表类似，但逻辑相反；它的内容是 from 时仅有的被复制出去的名字。 2. 如果模块的 name 变量是字符串'main'，意味着该文件正作为顶层脚本被执行，而不是被程序中的其他文件导入。也就是说，这个文件是被当作程序在用，而不是库。这个用途模式变量支撑了双模式代码与测试。 3.

通过模块的内置属性 `dict`。这是一个保存模块所有属性的普通字典，代码可以遍历它的键、值或键值对。需要时，属性值也可以按字符串名字索引 `dict` 获取，或调用 `getattr` 内置函数获取。

4. 用户输入通常以字符串形式进入脚本；要根据字符串名导入对应模块，你可以用 `exec` 构造并运行一条 `import` 语句，或者把字符串名传给 `import` 或 `importlib.importmodule` 函数。

5. 不行，Python 里没法从过去导入。我们可以安装（或固执地继续使用）旧版语言，但最新的 Python 通常就是最好的 Python（对在场各位 2.X 粉丝深表歉意）。

深度理解

·这 5 题就是全章知识骨架：X/all（名字导出控制）→ `name`（双模式）→ `dict`（自省）→ 动态导入（`exec/import`/`importlib`）→ `future`（语言演进哲学）。

·第 5 题是“送命题”也是“哲学题”：`future` 里的“未来”是官方已经决定要做的语义，只是按版本逐步默认化；“过去”没有任何机制能回溯。作者还借机调侃了 2.X 的怀旧党——既幽默又点出“语言向前演进”的现实。

·自测方式：遮住答案先自己写。尤其第 4 题要能写出三种动态导入手段并说出区别（`exec` 编译慢、`import` 返回最左组件、`importmodule` 返回最末组件），才算真正掌握。

25.17 第五部分练习 (Test Your Knowledge: Part V Exercises)

英文原文

See "Part V, Modules and Packages" in Appendix B for the solutions. 1. Import basics: Write a program that counts the lines and characters in a file (similar in spirit to part of what `wc` does on Unix). With your text editor, code a Python module called `mymod.py` that exports three top-level names:- A `countLines(name)` function that reads an input file specified by string `name`, and counts the number of lines in it (hint: `file.readlines()` does most of the work for you, and `len` does the rest, though you could count with `for` and file iterators to support massive files too).- A `countChars(name)` function that reads an input file and counts the number of characters in it (hint: `file.read()` returns a single string, which may be used in similar ways).- A `test(name)` function that calls both counting functions with a given input filename. Such a filename generally might be passed in, hardcoded, input from a user like you with the input built-in function, or pulled from a command line via the `sys.argv` list demoed in this chapter's `reloadall.py` example; for now, you can assume it's a passed-in function argument. All three `mymod` functions should expect a filename string to be passed in. If you type more than two or three lines per function, you're working much too hard—use the hints given! Next, test your module interactively, using `import` and attribute references to fetch your exports. Doe

s your PYTHONPATH need to include the directory where you created mymod.py? Try running your module on itself: for example, `test('mymod.py')`. Note that `test` opens the file twice; if you're feeling ambitious, you may be able to improve this by passing an open file object into the two count functions (hint: `file.seek(0)` is a file rewind).

2. `from/from \`: Test your mymod module from exercise 1 interactively by using `from` to load the exports directly, first by name, then using the `from` variant to fetch everything.

3. `\main\`: Add a line in your mymod module that calls the `test` function automatically only when the module is run as a script, not when it is imported. The line you add will probably test the value of `name` for the string `'main'`, as shown in this chapter. Try running your module from the system command line or other program-launch scheme; then, import the module and test its functions interactively. Does it still work in both modes?

4. Nested imports: Write a second module, `myclient.py`, that imports `mymod` and tests its functions; then run `myclient` from the system command line or other scheme. If `myclient` uses `from` to fetch from `mymod`, will `mymod`'s functions be accessible from the top level of `myclient`? What if it imports with `import` instead? Try coding both variations in `myclient` and test interactively by importing `myclient` and inspecting its `dict` attribute.

5. Package imports: Import your `mymod.py` file from a package. Create a subdirectory called `mypackage`

sted in a directory on your module import search path, copy or move the `mymod.py` module file you created in exercise 1 or 3 into the new directory, and try to import it with a package import of the form `import mypkg.mymod` and call its functions. Try to fetch your counter functions with a `from` too. This works on all Python platforms (that's part of the reason Python uses `"."` as a path separator). The package directory you create can be simply a subdirectory of the one you're working in; if it is, it will be found via the home directory component of the search path, and you won't have to configure your path. You also don't need an `init.py` file in the package directory your module was moved into to make this go, but make one with some basic prints in it and see if they run on each import or reload of the package folder. Finally, also copy `mymod.py` to the package folder's `main.py` and invoke it by running the folder itself; does it make sense to do that here? Can you still run the nested `mymod.py` module itself?

6. Reloads: Experiment with module reloads: if you haven't already, perform the tests in Chapter 23's `changer.py` (Example 23-10), changing the called function's message or behavior repeatedly, without stopping the Python REPL. Depending on your device, you might edit `changer` in another window, or suspend the Python interpreter and edit in the same window (on Unix, a `Ctrl+Z` key combination usually suspends the current process, and an `fg` command later resu

mes it, though a separate text-editor window can work just as well). 7. Circular imports: In the section on recursive (a.k.a. circular) import gotchas, importing `recur1` raised an error. But if you restart Python and import `recur2` interactively, the error doesn't occur—test this and see for yourself. Why do you think it works to import `recur2`, but not `recur1`? (Hint: Python records new modules before running their code, and later imports fetch the module first, whether the module is "complete" yet or not.) Now, run `recur1` as a top-level script file: `python3 recur1.py`. Do you get the same error that occurs when `recur1` is imported interactively? Why? (Hint: when modules are run as programs, they aren't imported, so this case has the same effect as importing `recur2` interactively; `recur2` is the first module imported.) What happens when you run `recur2` as a script? Circular imports are uncommon in practice. On the other hand, if you can understand why they are a potential problem, you know a lot about Python's import semantics.

中文翻译

答案见附录 B 的 "Part V, Modules and Packages" (第五部分：模块与包)。1. 导入基础：写一个程序，统计文件的行数和字符数（精神上类似于 Unix 上 `wc` 的部分功能）。用文本编辑器编写一个名为 `mymod.py` 的 Python 模块，导出三个顶层名字：- `countLines(name)` 函数：读取由字符串 `name` 指定的输入文件，统计其中行数（提示：`file`。

`readlines()` 帮你完成了大部分工作，`len` 搞定剩下的；不过你也可以用 `for` 和文件迭代器来计数，以支持超大文件）。

- `countChars(name)` 函数：读取输入文件并统计字符数（提示：`file.read()` 返回单个字符串，可用类似的方式处理）。
- `test(name)` 函数：用给定的输入文件名调用上面两个计数函数。这种文件名通常可能是传入的、硬编码的、用 `input` 内置函数向用户（像你这样的用户）收集的，或者通过 `sys.argv` 列表（本章 `reloadall.py` 示例演示过）从命令行取得；目前你可以假定它是传入的函数参数。三个 `mymod` 函数都应期望传入文件名（字符串）。如果每个函数写的代码超过两三行，说明你干得太费劲了——用给的那些提示！

接下来，交互式地测试你的模块，用 `import` 和属性引用来取用你的导出。你的 `PYTHONPATH` 需要包含创建 `mymod.py` 的目录吗？试试让模块对自己运行：例如 `test('mymod.py')`。注意 `test` 打开了文件两次；如果你有雄心，可以把一个打开的文件对象传给两个计数函数来改进它（提示：`file.seek(0)` 是文件回卷）。

2. `from/from \`：交互式测试练习 1 的 `mymod` 模块——用 `from` 直接加载导出：先按名字逐个导入，再用 `from` 变体一次性取全部。

3. `\main\`：在 `mymod` 模块里加一行，让 `test` 函数只在模块作为脚本运行、而不是被导入时自动调用。你要加的那行大概会像本章所示，把 `name` 与字符串 `'main'` 比较。试着从系统命令行或其他程序启动方式运行模块；然后再导入模块，交互式测试它的函数。两种模式都还能正常工作吗？

4. 嵌套导入：写第二个模块 `myclient.py`，导入 `mymod` 并测试它的函数；然后从系统命令行或其他方式运行 `myclient`。如果 `myclient` 用 `from` 从 `mymod` 取值，`mymod` 的函数能从 `myclient` 顶层直接访问吗？如果用 `import` 导入

呢？在 `myclient` 里试写两种变体，然后交互式导入 `myclient`，检查它的 `dict` 属性来验证。

5. 包导入：从包里导入你的 `mymod.py` 文件。在模块导入搜索路径上的某个目录里建一个名为 `mypkg` 的子目录，把练习 1 或 3 创建的 `mymod.py` 复制或移动进去，然后用 `import mypkg.mymod` 这种形式的包导入试试，并调用它的函数。也用 `from` 取一下计数函数。这在所有 Python 平台上都可行（这也是 Python 用 `"."` 作路径分隔符的部分原因）。你创建的包目录可以就是当前工作目录的子目录；如果是这样，它会通过搜索路径的 `home` 目录分量被找到，你就不用配置路径了。要让这个包工作，其实不需要在包目录里放 `init.py` 文件，不过你可以做一个、里面放几条基本 `print`，看看每次导入或重载这个包文件夹时它们是否运行。最后，把 `mymod.py` 复制为包文件夹里的 `main.py`，然后通过运行文件夹本身来调用它；在这里这么做有意义吗？你还能继续单独运行嵌套的 `mymod.py` 模块吗？

6. 重载：动手实验模块重载：如果还没做过，就完成第 23 章 `changer.py`（示例 23-10）里的测试——在不停止 Python REPL 的情况下，反复改变被调用函数的消息或行为。视设备情况，你可以在另一个窗口编辑 `changer`，或挂起 Python 解释器后在同一个窗口编辑（在 Unix 上，`Ctrl+Z` 组合键通常挂起当前进程，之后 `fg` 命令恢复它；当然，单独开一个文本编辑器窗口也一样好使）。

7. 循环导入：在递归（又叫循环）导入陷阱那一节，导入 `recur1` 报了错。但如果你重启 Python 后交互式导入 `recur2`，错误却不会出现——自己测试验证一下。为什么导入 `recur2` 可以、而 `recur1` 不行？（提示：Python 在运行模块代码之前就先登记模块，后续导入会先取到模块——无论模块那时“完整”与否。）现在，把 `recur1` 作

为顶层脚本文件运行：`python3 recur1.py`。你会得到与交互式导入 `recur1` 时一样的错误吗？为什么？（提示：模块作为程序运行时不是被导入的，所以这种情况与交互式导入 `recur2` 效果相同；`recur2` 是第一个被导入的模块。）你把 `recur2` 作为脚本运行又会怎样？循环导入在实践中并不常见。但另一方面，如果你能理解它们为什么可能成为问题，你对 Python 的导入语义就知之甚深了。

深度理解

·练习 1-3 是一条完整的主线：写工具模块 → 交互测试 → 加 `name` 双模式开关。做完这三步，你就亲手经历了本章 25.7、25.8 的全部要义。

·练习 4 的灵魂拷问：`from` 与 `import` 对“顶层可访问性”的影响——`from mymod import f` 让 `f` 成为 `myclient` 顶层名字，`import mymod` 则只有 `mymod` 一个顶层名。检查 `myclient.dict` 就是“看证据”的内省练习。

·练习 5 的隐藏考点：`init.py` 在 Python 3.3+ 是可选的（命名空间包），但放一个会执行顶层代码——观察“每次导入/重载包都运行”的副作用；`main.py` 让“运行文件夹”成为一个脚本入口（对应 25.7 的 NOTE）。

·练习 7 是整章最难也最有价值的一题：为什么 `import recur2` 不报错而 `import recur1` 报错？答：`recur2` 是第一个被导入的模块，它执行完自己再 `from recur1` 时，`recur1` 会从头执行到 `X=1`、`Y=2` 全部完成——拿到完整版。而 `import recur1` 时 `recur1` 半途导入 `recur2`，`recur2` 反向 `from recur1 import Y` 时 `recur1` 才执行到一半。

这题用上了"先登记后执行"与"部分初始化"两个机制，做完它，你就真正吃透了导入系统。

本章总结

技术拓展 (Technical Expansion)

- 实际项目中的应用场景
 - `name == 'main'`: 所有 CLI 工具、`manage.py`、`main.py` 的标准入口写法；库的自测代码也靠它隔离。
 - all 白名单: 任何对外发布库的 `init.py` 必备，让 `from pkg import`、IDE、`help()` 与文档工具都能识别"公开 API"。
 - `importlib.importmodule`: 插件系统、配置驱动加载、测试框架按字符串跑用例，几乎是唯一正解。
 - `reloadall` 思路: 热重载开发的雏形；现代框架 (FastAPI 的 `--reload`、调试器) 原理仍建立在"就地更新模块对象 + 属性查询"之上。
 - 循环导入规避: ORM 模型、事件注册中心、plugin 注册模式都用"延迟导入 + 注册表"来解耦。
 - 与 Java/C++ 的区别

维度	Python	Java / C++
模块/单元	模块 = 文件，命名空间即字典；无编译产物	编译单元 + 头文件 / package + class 文件
数据隐藏	约定 (X/all)，无强制	private/protected/internal 语言级强制
动态导入	<code>importlib.importmodule</code> ，运行时字动	<code>Class.forName/dlopen</code> ，反射框架

循环依赖	允许但需谨慎时序	链接期/编译期会报错，需前向声明或接口隔离
重载	运行时 <code>importlib.reload</code> (就地更新)	无从谈起，改完必须重编译重启动
内省	<code>dict</code> 、 <code>dir()</code> 、 <code>getattr</code> 原生即用	需反射库/宏，需额外设计

· Python 发展历史背景：future 是 2→3 转型遗留的"渐进演化"机制；reload 原先是内置函数，Python 3.4 后移入 `importlib` 模块；`sys.monitoring` 是 3.12 新增性能监控设施（也因此让 reloader 面临不可重载的情况）；模块级 `getattr/dir` (3.7 加入) 则把类的魔法能力第一次正式"下沉"到模块。这些年 `import` 系统从"简单的文件/目录查找"演进为"可插拔 Finder/Loader"体系（PEP 302、328、420 等），但"名字即命名空间字典键"的内省基石始终未变。

· 高级开发者需要掌握的相关知识：

· `inspect` 模块（比 `dir()/dict` 更完整的静态解析工具：`getmodule`、`getfile`、`getsource`）。

· `sys.modules` 的增删与副作用（测试隔离、热重载清理、防递归）。

· `importlib.abc` (Finder/Loader/MetaPathFinder) 与自定义导入器——可把"从 URL/数据库导入模块"做进自己的框架。

· 类型注解与 `getattr` 的动态接口在 `static type check` 中的冲突实践（如 `getattr` 返回 `Any`）。

· 缓存字节码 (`pycache`、`.pyc`) 与 `PYTHONHASHSEED/sys.dontwritebytecode` 的工程细节。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5)

· name 双模式、all/X、import as、dict 内省、importlib 动态导入——这五件事是日用知识，几乎每个 Python 项目都会碰到。

· reload、future、模块级 getattr、循环导入陷阱——更强的进阶，但 debug 和架构设计时能救命。

·应该掌握到什么程度：

·能脱口说出 from 是"复制非链接"；能解释 reload 为什么对 from X import f 无效。

·能手写 if name == 'main': main() 并理解为何是它而非别的。

·能手写一个"按名字字符串导入插件" (importlib 1 行) 和一个"传递式重载器" (递归 + dict + types.ModuleType) 。

·能凭感觉判断"该用 import module 还是 from module import name"。

·后续应该学习的内容：

1. 第 26 章起的类与面向对象 (Part VI) ——模块是"扁平命名空间"，类是"带继承搜索的命名空间"，dict 与属性解析会在这里升级为完整机制。

2. 第 30-31 章：getattr、setattr、dir 在调类中的完整版（属性拦截与委托）。

3. 第 36 章：unittest/doctest 正式测试工具——name 自测的"正规军"升级版。
4. 附录 A / Python 官方手册：importlib、sys、inspect 的深度参考。
5. 架构层面：看一个大项目（如 Django、FastAPI）的 modules 组织与 init.py 再导出约定，比对本章各知识点在真实代码里的落点。

至此，第 25 章与第五部分"模块与包"全部收尾。下一章起，进入 Python 面向对象的世界——类（class）。
